

UNIVERSITAT D'ALACANT

Escuela Politécnica Superior

Grado en Ingeniería Informática

TRABAJO DE FINAL DE GRADO

**Sistema de Trading Algorítmico Híbrido Basado en
Inteligencia Artificial y Procesamiento de Lenguaje
Natural**

Autor: Joan Romà Llorca

Tutor: José Ignacio Abreu Salas

Curso académico: 2025-2026

Fecha de entrega: Mayo 2026

Resumen

Se presenta el diseño, implementación y validación de un sistema de trading algorítmico híbrido para el mercado de futuros de criptomonedas. El sistema integra siete capas complementarias de análisis: un modelo de Machine Learning basado en LightGBM Regressor entrenado sobre 18 indicadores técnicos multi-temporalidad, un motor de Procesamiento de Lenguaje Natural que analiza el sentimiento de noticias financieras mediante FinBERT con reconocimiento de entidades por moneda, análisis de microestructura de mercado a través del Order Book Imbalance, filtros de tendencia macro basados en medias móviles y ADX en temporalidad horaria, un módulo de gestión de riesgo cuantitativa que implementa el Criterio de Kelly, stop loss dinámico basado en ATR y un Circuit Breaker de protección diaria, datos on-chain del Fear & Greed Index, y features multi-timeframe que cruzan información de 5 minutos con 1 hora.

La arquitectura sigue un patrón de microservicios orquestados con Docker Compose, compuesto por seis contenedores independientes: el bot principal de trading, el motor de análisis de sentimiento, el ingestor de datos on-chain, una base de datos de series temporales TimescaleDB, un dashboard Grafana y un servicio de monitorización MLOps con Tensorboard. Esta arquitectura permite la ejecución autónoma 24/7 con tolerancia a fallos.

El sistema se validó mediante backtesting histórico con metodología Walk-Forward sobre 11 criptomonedas durante múltiples periodos que abarcan distintos regímenes de mercado. Cabe señalar que, durante el período de backtesting, la capa de análisis de sentimiento NLP (Capa 5) operó en modo fallback con puntuación neutra constante (score = 0.0), dado que las fuentes RSS históricas no se encontraban disponibles; por tanto, los resultados corresponden efectivamente a las seis capas técnicas y de gestión de riesgo del sistema. En el escenario alcista, se obtuvo un Win Rate del 53.3% con un Max Drawdown contenido del 2.97% y un rendimiento neto del +2.57%. En el escenario de crash, donde el mercado cayó un 34.57%, el sistema limitó las pérdidas a un 1.03%, generando un alpha de +33.54 puntos porcentuales. En el escenario bajista (-36%), las pérdidas alcanzaron un -7.00%, con un alpha de +29.00 puntos porcentuales. El alpha promedio sobre Buy & Hold fue de +9.83 puntos porcentuales. El análisis de explicabilidad mediante SHAP reveló que los indicadores de volumen y volatilidad son los predictores más relevantes, por encima de indicadores de momentum tradicionales.

Se desarrolló adicionalmente una suite de tests unitarios con Pytest, un script de despliegue automatizado y documentación técnica completa. El sistema se encuentra actualmente en fase de Forward-Testing con datos de mercado en tiempo real.

Palabras clave: trading algorítmico, aprendizaje automático, procesamiento de lenguaje natural, microservicio, gestión de riesgo

Abstract

This work presents the design, implementation, and validation of a hybrid algorithmic trading system for the cryptocurrency futures market. The system integrates seven complementary analysis layers: a Machine Learning model based on LightGBM Regressor trained on 18 multi-timeframe technical indicators, a Natural Language Processing engine that analyzes the sentiment of financial news using FinBERT with per-coin Named Entity Recognition, market microstructure analysis through Order Book Imbalance, macro trend filters based on hourly moving averages and ADX, a quantitative risk management module implementing the Kelly Criterion with ATR-based dynamic stop loss and a daily Circuit Breaker, on-chain sentiment signals via the Fear & Greed Index, and multi-timeframe features crossing 5-minute and 1-hour information.

The system architecture follows a microservices pattern orchestrated with Docker Compose, comprising six independent containers: the main trading bot, the sentiment analysis engine, the on-chain data ingestor, a TimescaleDB time-series database, a Grafana dashboard, and a Tensorboard MLOps monitoring service. This architecture enables autonomous 24/7 operation with fault tolerance.

The system was validated through historical backtesting using Walk-Forward methodology across 11 cryptocurrencies over multiple periods covering different market regimes. It should be noted that during the backtesting period, the NLP sentiment analysis layer (Layer 5) operated in fallback mode with a constant neutral score (0.0), as historical RSS feeds were unavailable for the evaluated period; therefore, the reported results correspond to the six technical and risk management layers of the system. In the bullish scenario, a Win Rate of 53.3% was achieved with a contained 2.97% Max Drawdown and a net positive return of +2.57%. During the crash scenario, where the market declined 34.57%, the system limited losses to just 1.03%, generating an alpha of +33.54 percentage points. In the bear scenario (-36%), losses reached -7.00%, with an alpha of +29.00 percentage points. On average, the system generated an alpha of +9.83 percentage points over Buy & Hold. SHAP-based model explainability analysis revealed that volume and volatility indicators are the most relevant predictors, ranking above traditional momentum indicators.

Additionally, a Pytest unit test suite, an automated deployment script, and comprehensive technical documentation were developed. The system is currently undergoing Forward-Testing with live market data.

Keywords: algorithmic trading, machine learning, natural language processing, microservice, risk management

ÍNDICE

Resumen.....	2
Abstract.....	4
Agradecimientos	12
Capítulo 1 - Introducción	13
1.1 Motivación y Contexto del Proyecto	13
1.2 Objetivo General.....	14
1.3 Objetivos Específicos	14
1.4 Hipótesis de la Investigación.....	15
1.5 Alcance y Limitaciones.....	15
1.5.1 Alcance.....	15
1.5.2 Limitaciones	16
1.5.3 Limitaciones Metodológicas Adicionales	16
Capítulo 2 - Estado del Arte.....	18
2.1 Trading Algorítmico: Definición, Historia y Evolución.....	18
2.1.1 Breve Historia.....	18
2.1.2 Categorías de Estrategias.....	19
2.2 Machine Learning Aplicado a Finanzas	19
2.2.1 Modelos Supervisados: Regresión vs Clasificación	19
2.2.2 Gradient Boosted Decision Trees.....	20
2.2.3 Redes Neuronales vs Modelos de Ensemble.....	20
2.3 Procesamiento de Lenguaje Natural en Finanzas.....	21
2.3.1 Análisis de Sentimiento Financiero.....	21
2.3.2 Comparativa de Modelos NLP.....	22
2.3.3 Named Entity Recognition (NER) en Finanzas	22
2.4 Gestión de Riesgo Cuantitativa	23
2.4.1 Criterio de Kelly y Half-Kelly.....	23
2.4.2 Stop Loss Dinámico Basado en ATR.....	23
2.4.3 Métricas de Rendimiento	24
2.5 Infraestructura de Despliegue: Docker y Microservicios	24
2.6 Framework Freqtrade y FreqAI	25
2.7 Trabajos Relacionados.....	26
Capítulo 3 - Análisis de Requisitos.....	27
3.1 Requisitos Funcionales.....	27
3.2 Requisitos No Funcionales	28

3.3 Restricciones del Sistema	28
3.4 Universo de Activos.....	29
3.5 Metodología de Desarrollo.....	29
3.6 Planificación Temporal	30
3.7 Estimación de Costes	31
3.8 Análisis de Riesgos	32
Capítulo 4 - Diseño del Sistema.....	34
4.1 Visión General del Modelo.....	34
4.2 Motor de Decisión Multi-Capa.....	35
4.2.1 Ciclo de Vida de una Operación.....	36
4.3 Componente de Inteligencia Artificial (LightGBM)	37
4.3.1 Feature Engineering: De 5 a +18 Features	37
4.3.2 Codificación Temporal Cíclica.....	39
4.3.3 Target de Regresión y Horizonte Temporal.....	39
4.3.4 Re-entrenamiento Continuo (Walk-Forward).....	40
4.4 Componente NLP (FinBERT + NER).....	40
4.4.1 Pipeline de Procesamiento.....	40
4.4.2 NLP como Filtro, no como Generador.....	41
4.4.3 Fallback y Resiliencia.....	41
4.5 Gestión de Riesgo Cuantitativa	42
4.5.1 Kelly empírico al 40% ('custom_stake_amount')	42
4.5.2 Stop Loss Dinámico Basado en ATR.....	43
4.5.3 Trailing Stop Híbrido	43
4.5.4 Circuit Breaker ('confirm_trade_entry').....	43
4.5.5 Filtro On-Chain (Fear & Greed).....	44
4.6 Pipeline On-Chain	44
Capítulo 5 - Solución Propuesta: Bot y Aplicación	45
5.1 Arquitectura General de Microservicios	45
5.2 Flujo de Datos entre Servicios.....	46
5.2.1 Mecanismos de Sincronización y Coherencia de Datos	47
5.2.2 Política de Reinicio y Tolerancia a Fallos	48
5.3 Entorno de Desarrollo.....	48
5.4 Implementación de los Componentes Principales	48
5.4.1 Señales de Entrada ('populate_entry_trend').....	48
5.4.2 Señales de Salida	49
5.5 Motor NLP ('sentiment_ingestor.py').....	49
5.6 Motor On-Chain ('onchain_ingestor.py')	50

5.7 Dockerización y Orquestación.....	50
5.7.1 Dockerfile Principal.....	50
5.7.2 Gestión de Secretos.....	50
5.8 Pipeline MLOps.....	51
5.9 Diseño de la Base de Datos (TimescaleDB).....	51
<i>Capítulo 6 - Evaluación del Modelo de Trading.....</i>	<i>52</i>
6.1 Metodología Experimental: Walk-Forward Analysis	52
6.1.1 Limitaciones del Motor de Backtesting.....	53
6.2 Escenarios de Mercado	53
6.3 Resultados por Escenario	55
6.3.1 Escenario 1: Bull Market (Abril – Mayo 2025)	55
6.3.2 Escenario 2: Crash (Octubre – Diciembre 2025).....	56
6.3.3 Escenario 3: Mercado Lateral (Enero – Marzo 2025)	57
6.3.4 Escenario 4: Bear Market (Julio – Octubre 2025)	58
6.4 Tabla Consolidada de Resultados	59
6.5 Comparativa Bot vs Buy & Hold.....	60
6.5.1 Análisis de Significancia Estadística	61
6.6 Explicabilidad del Modelo (SHAP + Feature Importance)	62
6.6.1 Consideraciones Metodológicas sobre la Interpretabilidad SHAP	63
<i>Capítulo 7 - Evaluación del Sistema</i>	<i>64</i>
7.1 Suite de Tests Unitarios (Pytest).....	64
7.2 Validación de Requisitos No Funcionales	65
7.2.1 Rendimiento.....	65
7.2.2 Disponibilidad y Tolerancia a Fallos.....	65
7.2.3 Portabilidad y Despliegue.....	65
7.2.4 Seguridad.....	65
7.3 Validación de Objetivos Específicos.....	66
<i>Capítulo 8 - Discusión de Resultados</i>	<i>67</i>
8.1 Interpretación de los Resultados por Escenario.....	67
8.1.1 Limitaciones del Benchmark.....	68
8.2 Comparación con Trabajos Relacionados	68
8.3 Validación de la Hipótesis	69
8.4 Fortalezas del Sistema	70
8.5 Debilidades y Factores Externos	70
<i>Capítulo 9 - Evolución del Sistema y Problemas Resueltos</i>	<i>72</i>
9.1 Línea Temporal de Desarrollo.....	72

9.2 Conflicto de Precedencia Config vs Strategy (v1.0).....	73
9.3 Limitación Artificial de Stake (v1.0)	73
9.4 Feature Engineering: De Binario a Regresión (v2.0).....	73
9.5 Bug del HTML en NLP	74
9.6 Parche de Datasieve 0.1.9	74
9.7 Optimización de PyTorch CPU-Only	74
Capítulo 10 - Conclusiones y Trabajo Futuro	76
10.1 Conclusión General.....	76
10.2 Conclusiones Específicas	76
10.3 Aportaciones del Trabajo.....	77
10.4 Lecciones Aprendidas.....	78
10.5 Trabajo Futuro	79
10.5.1 Reinforcement Learning (DQN)	79
10.5.2 Fuentes de Datos Adicionales	79
10.5.3 Multi-Exchange y Multi-Activo	80
10.5.4 Implementación de Order Book Imbalance Real	80
10.5.5 Análisis Ablativo	80
Anexos.....	84
Anexo A - Código Fuente de la Estrategia Principal	84
Anexo B - Configuración Docker Compose	86
Anexo C - Guía de Despliegue.....	87
Anexo D - Repositorio Completo	88

ÍNDICE DE TABLAS

Tabla 1: Categorías de estrategias de trading algorítmico	19
Tabla 2: Comparativa de frameworks de Gradient Boosting	20
Tabla 3: Comparativa de modelos de análisis de sentimiento financiero	22
Tabla 4: Métricas estándar de evaluación de estrategias de trading	24
Tabla 5: Requisitos funcionales del sistema	27
Tabla 6: Requisitos no funcionales del sistema	28
Tabla 7: Universo de activos monitorizados	29
Tabla 8: Planificación temporal del proyecto	30
Tabla 9: Estimación de costes del proyecto	31
Tabla 10: Análisis de riesgos y mitigaciones	32
Tabla 11: Reglas de las 7 capas de confluencia (señal LONG)	35
Tabla 12: Features del modelo organizadas por categoría	38
Tabla 13: Servicios del sistema y configuración de red	46
Tabla 14: Stack tecnológico del entorno de desarrollo	48
Tabla 15: Tabla comparativa de los períodos seleccionados	54
Tabla 16: Resultados del backtesting - Bull Market	55
Tabla 17: Resumen consolidado de métricas por escenario	59
Tabla 18: Comparativa global Bot vs Buy & Hold	60
Tabla 19: Distribución de importancia por categoría	62
Tabla 20: Resultados de la suite de tests unitarios (10/10 PASSED en 0.24s)	64
Tabla 21: Validación de cumplimiento de objetivos específicos	66
Tabla 22: Evolución del sistema - Problemas y soluciones	72

ÍNDICE DE ILUSTRACIONES

Ilustración 1: Motor de Decisión Multi-Capa - 7 capas de confluencia	35
Ilustración 2: Ciclo de vida de una operación de trading	37
Ilustración 3: Capas de Gestión de Riesgo del sistema	42
Ilustración 4: Arquitectura de microservicios del sistema.....	45
Ilustración 5: Rendimiento Bot vs Buy & Hold por escenario	55
Ilustración 6: Alpha generado por escenario sobre Buy & Hold.....	61
Ilustración 7: Importancia de features por categoría (SHAP)	62
Ilustración 8: Evolución del sistema v1.0 → v3.0.....	72

Agradecimientos

A mi familia, por su apoyo incondicional durante toda la carrera universitaria y especialmente durante el desarrollo de este proyecto, que ha requerido innumerables horas de trabajo frente a la pantalla.

A mi tutor, José Ignacio Abreu Salas, por su orientación académica y por confiar en un proyecto que combina disciplinas tan diversas como la inteligencia artificial, las finanzas cuantitativas y la ingeniería de software.

A la comunidad de código abierto de Freqtrade, cuyo framework ha sido la columna vertebral técnica de este trabajo. Sin su documentación, su código y su comunidad activa en Discord, este proyecto no habría sido posible.

A la Universitat d'Alacant, por proporcionarme los conocimientos y las herramientas necesarias para abordar un reto de esta complejidad.

Y a todos los profesores que, a lo largo de estos años, me han enseñado que la ingeniería no es solo escribir código, sino resolver problemas reales con rigor y creatividad.

Capítulo 1 - Introducción

1.1 Motivación y Contexto del Proyecto

El mercado de criptomonedas opera las 24 horas del día, los 7 días de la semana, sin festivos ni pausas. A diferencia de los mercados bursátiles tradicionales como el NYSE o el IBEX 35, que cierran por las noches y los fines de semana, los exchanges de criptomonedas como Binance procesan transacciones de forma ininterrumpida. Esta naturaleza continua supone un desafío insalvable para el operador humano: resulta físicamente imposible monitorizar el mercado de forma permanente sin incurrir en fatiga cognitiva, errores emocionales y oportunidades perdidas.

La industria del trading cuantitativo, históricamente reservada a fondos de cobertura (*hedge funds*) como Renaissance Technologies, Two Sigma o Citadel, ha experimentado una democratización progresiva gracias a la aparición de frameworks de código abierto como Freqtrade, Backtrader y Zipline (Jansen, 2020). Estas herramientas permiten a desarrolladores independientes diseñar, probar y desplegar estrategias de trading algorítmico sin necesidad de infraestructura propietaria valorada en millones de euros.

Paralelamente, los avances recientes en aprendizaje automático (*Machine Learning*) y procesamiento de lenguaje natural (*Natural Language Processing*) han abierto nuevas vías para el análisis de mercados financieros. Los modelos de *Gradient Boosted Decision Trees* como LightGBM (Ke et al., 2017) han demostrado un rendimiento superior a las redes neuronales profundas en datos tabulares estructurados (Grinsztajn et al., 2022), mientras que modelos especializados como FinBERT (Araci, 2019) permiten cuantificar el sentimiento de noticias financieras con una precisión cercana al 87%.

El presente Trabajo de Final de Grado nace de la confluencia de tres disciplinas de la Ingeniería Informática:

1. **Inteligencia Artificial y Machine Learning:** Uso de modelos de *Gradient Boosted Decision Trees* (LightGBM) para predecir movimientos de precio a corto plazo con precisión cuantificable.
2. **Procesamiento de Lenguaje Natural (NLP):** Integración de un modelo pre-entrenado de análisis de sentimiento financiero (FinBERT) que procesa noticias en tiempo real para filtrar señales de trading según el estado del mercado.

3. **Ingeniería de Software y DevOps:** Diseño de una arquitectura de microservicios orquestada con Docker Compose, con persistencia en bases de datos de series temporales (TimescaleDB), visualización en Grafana y notificaciones en tiempo real vía Telegram.

La motivación subyacente no es meramente académica. El sistema desarrollado aspira a demostrar empíricamente que un agente de software, dotado de las herramientas analíticas adecuadas, puede generar rendimientos positivos ajustados a riesgo en el mercado de criptomonedas, incluso durante periodos de tendencia bajista, superando así a la estrategia pasiva de *Buy & Hold*.

1.2 Objetivo General

Diseñar, implementar y validar un sistema de trading algorítmico híbrido que combine técnicas de Machine Learning, Procesamiento de Lenguaje Natural y análisis técnico avanzado para operar de forma autónoma en el mercado de futuros de criptomonedas, maximizando la rentabilidad ajustada a riesgo.

1.3 Objetivos Específicos

- **OE1 - Motor de Inteligencia Artificial:** Implementar un modelo de regresión basado en LightGBM capaz de predecir el porcentaje de cambio del precio de un activo en las próximas N velas, alimentado por un pipeline de 18 features técnicas multi-timeframe.
- **OE2 - Motor de Procesamiento de Lenguaje Natural:** Desarrollar un microservicio independiente que descargue titulares de noticias financieras en tiempo real (RSS), los analice con el modelo FinBERT de HuggingFace con Named Entity Recognition per-coin, y almacene los resultados de sentimiento en una base de datos de series temporales.
- **OE3 - Gestión de Riesgo Cuantitativa:** Implementar mecanismos de protección de capital basados en el Criterio de Kelly empírico para el dimensionamiento dinámico de posiciones, stop loss adaptativo basado en el Average True Range (ATR), un Circuit Breaker que detenga la operativa si las pérdidas diarias superan un umbral predefinido, y filtros on-chain basados en el Fear & Greed Index.
- **OE4 - Arquitectura de Microservicios:** Diseñar una infraestructura Dockerizada compuesta por 6 contenedores independientes (bot principal, motor NLP, ingestor on-

chain, base de datos TimescaleDB, dashboard Grafana, MLOps Tensorboard) que permita la ejecución autónoma 24/7.

- **OE5 - Validación Empírica:** Ejecutar backtests históricos con metodología Walk-Forward sobre 11 criptomonedas durante múltiples periodos y regímenes de mercado (alcista, bajista, lateral, crash), calcular métricas profesionales de rendimiento (Sharpe Ratio, Sortino Ratio, Max Drawdown, Win Rate) y analizar la explicabilidad del modelo mediante SHAP.
- **OE6 - Calidad de Software:** Desarrollar una suite de tests unitarios con Pytest que valide matemáticamente la lógica del Criterio de Kelly, el stop loss dinámico ATR y el pipeline de procesamiento NLP.

1.4 Hipótesis de la Investigación

Un sistema de trading algorítmico que combine IA y NLP puede generar rendimientos positivos ajustados a riesgo en criptomonedas, un ratio Sharpe superior a 1 y un Max Drawdown inferior al 15%, superando la estrategia pasiva de Buy & Hold en mercados bajistas.

1.5 Alcance y Limitaciones

1.5.1 Alcance

El sistema abarca el ciclo completo de vida de un bot de trading algorítmico:

- **Ingesta de datos:** Descarga automática de datos de mercado (velas OHLCV) desde la API de Binance y titulares de noticias desde feeds RSS (CoinDesk, CoinTelegraph, Bitcoin.com).
- **Procesamiento:** Cálculo de 18 indicadores técnicos multi-timeframe, análisis de sentimiento NLP per-coin, entrenamiento del modelo de IA con ventana deslizante de 30 días.
- **Decisión:** Motor de 7 capas que requiere confluencia de todos los filtros para emitir una señal de compra o venta.
- **Ejecución:** Envío de órdenes al exchange (modo simulado en la fase de validación del proyecto).
- **Monitorización:** Dashboard visual (Grafana + FreqUI), métricas MLOps (Tensorboard) y alertas en tiempo real (Telegram).

1.5.2 Limitaciones

- El sistema opera en **modo simulado** (*dry-run*) durante la fase de desarrollo y validación. La operativa con dinero real requiere un periodo adicional de Forward-Testing de al menos 30 días, actualmente en curso.
- Los datos de sentimiento NLP se limitan a titulares en **inglés** procedentes de 3 fuentes RSS. No se analizan redes sociales (Twitter/X) ni foros (Reddit).
- El modelo de IA no incorpora datos macroeconómicos externos (tipos de interés, inflación, decisiones de la Fed).
- Los resultados del backtest, aunque consistentes en múltiples escenarios, no garantizan rendimientos futuros debido a la naturaleza estocástica de los mercados financieros.
- El sistema está optimizado para el mercado de criptomonedas y no se ha validado en mercados de renta variable tradicional.
- En escenarios con pocas operaciones (7 trades en Bear Market), la muestra es estadísticamente insuficiente para extraer conclusiones con alta confianza. No se han calculado intervalos de confianza ni tests de significancia estadística.
- El sistema opera sobre 11 criptomonedas simultáneamente sin un mecanismo explícito de gestión de la correlación entre activos. En escenarios de crash sistémico, donde todas las criptomonedas caen simultáneamente, las pérdidas en múltiples posiciones podrían acumularse antes de que el Circuit Breaker actúe.

1.5.3 Limitaciones Metodológicas Adicionales

El modelado predictivo de series temporales financieras presenta desafíos estadísticos fundamentales que este trabajo asume como limitaciones intrínsecas:

- **No-estacionariedad:** Las distribuciones estadísticas de las variables financieras (*features*) mutan a lo largo del tiempo, exhibiendo fenómenos complejos como el agrupamiento de volatilidad (*volatility clustering*) o el cambio de regímenes de reversión a la media (*mean reversion regimes*). Esta inestabilidad estructural limita severamente la capacidad de generalización de cualquier modelo supervisado entrenado sobre ventanas históricas.
- **Concept Drift:** Derivado de la no-estacionariedad, el modelo padece de deriva conceptual. Un algoritmo entrenado bajo un régimen de mercado específico perderá eficacia

predictiva al aplicarse sobre un régimen diferente. Aunque la arquitectura del bot mitiga parcialmente este riesgo mediante un re-entrenamiento continuo de los modelos LightGBM cada 2 horas, esta técnica reduce la latencia de adaptación pero no elimina el problema de raíz.

- **Sesgo de Supervivencia:** El universo de inversión consta de 11 criptomonedas seleccionadas por su alta capitalización y liquidez vigentes en abril de 2026. Sin embargo, evaluar el rendimiento histórico de este ecosistema sobre datos de 2025 introduce un sesgo de supervivencia evidente: elude contabilizar aquellos activos que, gozando de alta capitalización en el pasado, sufrieron colapsos o desapariciones antes de 2026. Al evaluar exclusivamente a los “ganadores” retrospectivos, las métricas de *backtesting* presentan un sesgo inevitable de optimismo.

Para abordar estadísticamente estas restricciones, se propone como trabajo futuro la aplicación sistemática de pruebas de raíces unitarias (Test Dickey-Fuller Aumentado o ADF) sobre cada *feature* antes de su ingesta, garantizando la estacionariedad de las señales predictivas.

Capítulo 2 - Estado del Arte

2.1 Trading Algorítmico: Definición, Historia y Evolución

El trading algorítmico se define como la ejecución automatizada de órdenes de compra y venta en mercados financieros mediante algoritmos informáticos que siguen reglas predefinidas basadas en variables de tiempo, precio, volumen u otras métricas cuantitativas (Aldridge, 2013).

2.1.1 Breve Historia

Los orígenes del trading algorítmico se remontan a la década de 1970, cuando la Bolsa de Nueva York introdujo el sistema DOT (*Designated Order Turnaround*), permitiendo el envío electrónico de órdenes. Sin embargo, el verdadero punto de inflexión llegó en 1998, cuando la SEC (*Securities and Exchange Commission*) de Estados Unidos aprobó el uso de exchanges electrónicos, abriendo la puerta a los *Electronic Communication Networks* (ECNs).

En la década de 2000, el trading de alta frecuencia (*High-Frequency Trading*, HFT) se convirtió en la fuerza dominante de los mercados, representando más del 60% del volumen negociado en bolsas estadounidenses (Hendershott et al., 2011). Firmas como Renaissance Technologies, con su fondo Medallion, demostraron que los algoritmos cuantitativos podían generar retornos anualizados superiores al 66% durante más de tres décadas (Zuckerman, 2019).

La democratización del trading algorítmico se aceleró con la aparición de APIs públicas de exchanges como Binance (2017), que permiten a cualquier desarrollador con conocimientos de programación acceder a datos de mercado en tiempo real y ejecutar órdenes de forma programática. Frameworks de código abierto como Freqtrade, Backtrader y Zipline han reducido la barrera de entrada, permitiendo que desarrolladores independientes construyan sistemas que, hace una década, habrían requerido equipos de ingeniería de decenas de personas (Jansen, 2020).

2.1.2 Categorías de Estrategias

Tabla 1. Categorías de estrategias de trading algorítmico

Categoría	Horizonte Temporal	Ejemplo	Infraestructura
High-Frequency Trading (HFT)	Microsegundos – milisegundos	Arbitraje estadístico	Co-location en exchanges
Scalping	Segundos – minutos	Mean reversion intradía	Servidores de baja latencia
Swing Trading	Horas – días	Seguimiento de tendencia	VPS o servidor dedicado
Position Trading	Semanas – meses	Momentum macro	Ejecución manual asistida

El sistema desarrollado en este TFG se clasifica como **Swing Trading Intradía Computarizado**: las operaciones tienen una duración media de 3 a 8 horas, buscando capturar movimientos de tendencia confirmados por múltiples indicadores en la temporalidad de 5 minutos con contexto horario.

2.2 Machine Learning Aplicado a Finanzas

2.2.1 Modelos Supervisados: Regresión vs Clasificación

En el contexto del trading algorítmico, el Machine Learning supervisado se aplica fundamentalmente en dos modalidades:

- **Clasificación binaria:** El modelo predice si el precio subirá (1) o bajará (0) en un horizonte temporal definido. Es conceptualmente simple pero pierde información sobre la magnitud del movimiento, imposibilitando el dimensionamiento proporcional de las posiciones.
- **Regresión continua:** El modelo predice el porcentaje exacto de cambio del precio. Permite al sistema distinguir entre movimientos marginales (+0.01%) y movimientos significativos (+2%), habilitando un dimensionamiento de posición proporcional a la confianza de la predicción.

Investigaciones recientes han demostrado que los modelos de regresión, cuando se combinan con umbrales de filtrado adaptativos, superan a los clasificadores binarios en entornos de trading con costes de transacción no nulos (Krauss et al., 2017). Este TFG implementa un

target de regresión, dado que permite una toma de decisiones más sofisticada que el enfoque binario tradicional.

2.2.2 Gradient Boosted Decision Trees

Los modelos de *Gradient Boosting* construyen un ensamble de árboles de decisión de forma secuencial, donde cada nuevo árbol corrige los errores residuales del anterior. La función objetivo se minimiza iterativamente mediante descenso de gradiente en el espacio funcional (Friedman, 2001). Las tres implementaciones más populares son:

Tabla 2: Comparativa de frameworks de Gradient Boosting

Framework	Autor	Velocidad	Uso de Memoria	Crecimiento del Árbol
XGBoost	Chen y Guestrin (2016)	Alta	Alta	Level-wise
LightGBM	Ke et al. (2017)	Muy alta	Baja	Leaf-wise
CatBoost	Prokhorenkova et al. (2018)	Media	Media	Symmetric

LightGBM fue seleccionado para este proyecto por tres razones fundamentales:

1. **Velocidad de entrenamiento:** Utiliza un esquema de crecimiento de árboles *leaf-wise* (en vez de *level-wise*), lo que reduce el tiempo de entrenamiento hasta un orden de magnitud frente a XGBoost en datasets grandes.
2. **Eficiencia en memoria:** Emplea *Gradient-based One-Side Sampling* (GOSS) y *Exclusive Feature Bundling* (EFB) para reducir el número de muestras y features sin pérdida significativa de precisión (Ke et al., 2017).
3. **Integración nativa con FreqAI:** El framework Freqtrade proporciona soporte nativo para LightGBM a través de su módulo FreqAI, simplificando el pipeline de entrenamiento, predicción y re-entrenamiento continuo.

2.2.3 Redes Neuronales vs Modelos de Ensamble

Aunque las redes neuronales profundas (LSTM, Transformers) han demostrado resultados prometedores en predicción de series temporales financieras (Fischer y Krauss, 2018), los

modelos de ensamble basados en árboles presentan ventajas pragmáticas para el trading algorítmico en producción:

- **Interpretabilidad:** Un árbol de decisión es explicable mediante técnicas como SHAP (Lundberg y Lee, 2017); una red neuronal profunda es una caja negra.
- **Datos tabulares:** Los árboles de decisión son superiores a las redes neuronales en datos tabulares estructurados (Grinsztajn et al., 2022), que es precisamente el formato de los indicadores técnicos.
- **Tiempo de entrenamiento:** Un modelo LightGBM se entrena en segundos; un Transformer financiero puede tardar horas en GPU.
- **Robustez ante overfitting:** Con hiperparámetros adecuados y validación Walk-Forward, LightGBM generaliza mejor que las redes neuronales en datasets financieros de tamaño moderado (Shwartz-Ziv y Armon, 2022).

2.3 Procesamiento de Lenguaje Natural en Finanzas

2.3.1 Análisis de Sentimiento Financiero

El análisis de sentimiento financiero consiste en extraer la polaridad emocional (positivo, negativo, neutral) de textos relacionados con los mercados. La investigación de Tetlock (2007) encontró correlaciones estadísticamente significativas entre el pesimismo en medios financieros y movimientos posteriores en los precios de acciones del S&P 500, aunque la robustez de estos resultados fuera de la muestra original ha sido objeto de debate en la literatura posterior. No obstante, este trabajo estableció un precedente para la integración de datos textuales en modelos cuantitativos.

Estudios posteriores han confirmado que el sentimiento de noticias financieras tiene poder predictivo significativo en horizontes temporales de minutos a días, especialmente en mercados de criptomonedas donde la información se propaga más rápidamente que en mercados regulados (Chen et al., 2020).

2.3.2 Comparativa de Modelos NLP

Tabla 3: Comparativa de modelos de análisis de sentimiento financiero

Modelo	Base	Dominio	Precisión	Coste	Latencia
VADER	Reglas léxicas	General	~65%	Gratuito	<1ms
FinBERT	BERT (Google)	Financiero	86–97%	Gratuito	~5ms
GPT-4	Transformer	General	~90%	API de pago	~500ms
BloombergGPT	Transformer	Financiero	~91%	No público	N/A

FinBERT (Araci, 2019) fue seleccionado por su equilibrio entre **precisión** (86–97% en benchmarks financieros según el nivel de acuerdo entre anotadores), **velocidad** (inferencia en ~5ms por texto en CPU) y **coste** (gratuito, sin necesidad de API de pago). Está basado en la arquitectura BERT (Devlin et al., 2019) y fue fine-tuned sobre el Financial PhraseBank (Malo et al., 2014), corpus de 4.840 frases procedentes de noticias financieras. La precisión reportada varía según el nivel de acuerdo entre anotadores: 97% en el subconjunto de consenso total y 86% en el dataset completo (Araci, 2019).

Nota sobre la precisión reportada: La precisión del 97% de FinBERT corresponde al subconjunto del corpus Financial PhraseBank con acuerdo unánime entre todos los anotadores. El valor del 86% corresponde al dataset completo incluyendo casos ambiguos. En titulares RSS de producción, la precisión esperada es más cercana al rango inferior dado que el dominio de las fuentes (CoinDesk, CoinTelegraph) difiere del corpus de entrenamiento original.

2.3.3 Named Entity Recognition (NER) en Finanzas

El reconocimiento de entidades nombradas permite identificar qué criptomoneda específica se menciona en cada titular, habilitando un análisis de sentimiento *per-coin* en lugar de un sentimiento global del mercado. Este enfoque es particularmente relevante en el mercado de criptomonedas, donde la correlación entre activos no es constante y un titular negativo sobre una moneda específica puede no afectar al resto del universo de activos.

2.4 Gestión de Riesgo Cuantitativa

2.4.1 Criterio de Kelly y Half-Kelly

El Criterio de Kelly (Kelly, 1956) es una fórmula matemática que determina la fracción óptima del capital a arriesgar en cada apuesta para maximizar el crecimiento geométrico a largo plazo:

$$f^* = \frac{bp - q}{b}$$

Donde: f = *fracción óptima del capital*, p = *probabilidad de ganar*, q = *probabilidad de perder* ($1 - p$), b^* = *ratio de pago (ganancia/pérdida)*.

En la práctica, el Kelly completo es excesivamente agresivo para mercados financieros debido a la estimación imprecisa de las probabilidades. La variante **Half-Kelly** (50% del Kelly completo) es el estándar en la industria de hedge funds (Thorp, 2006), ya que reduce la volatilidad del portfolio en un 50% a cambio de sacrificar solo un 25% de la tasa de crecimiento esperada.

Una alternativa pragmática a ambas variantes es el denominado **Kelly empírico**: en lugar de calcular f^* a partir de estimaciones probabilísticas, que en mercados financieros son inherentemente imprecisas, se fija directamente una fracción de capital basada en experiencia histórica y se ajusta dinámicamente según la confianza del modelo y las condiciones de volatilidad. Este enfoque sacrifica la optimalidad teórica a cambio de robustez ante errores de estimación, y es el adoptado en este trabajo con un parámetro base del 40%.

Este TFG implementa un **Kelly empírico al 40%**, un parámetro calibrado empíricamente durante el proceso de Hyperopt que actúa como cota superior de la fracción de capital a arriesgar por operación.

2.4.2 Stop Loss Dinámico Basado en ATR

El Average True Range (ATR) es un indicador de volatilidad desarrollado por Wilder (1978) que mide el rango medio de movimiento del precio en N periodos. A diferencia de un stop loss fijo (por ejemplo, -1%), un stop basado en ATR se adapta automáticamente a las condiciones del mercado:

- **Mercado volátil (ATR alto):** El stop se amplía para evitar que el ruido del mercado active una salida prematura.
- **Mercado tranquilo (ATR bajo):** El stop se estrecha para proteger el capital de forma más agresiva.

La fórmula implementada en este TFG es: $stop = -(2 \times ATR_{14}) / precio_actual$. Dado que el stop loss se define matemáticamente como un porcentaje negativo, los límites de seguridad se aplican mediante la expresión $\max(\min(stop, -0.005), -0.03)$. En el dominio de los números negativos, la función $\min(\dots, -0.005)$ asegura que el stop no sea más ajustado que el -0.5% (piso mínimo), mientras que $\max(\dots, -0.03)$ garantiza que no sea más amplio que el -3.0% (techo máximo de pérdida permitida). Esta formulación evita ambigüedades en la gestión del riesgo dinámico.

2.4.3 Métricas de Rendimiento

Tabla 4: Métricas estándar de evaluación de estrategias de trading

Métrica	Fórmula	Interpretación
Sharpe Ratio	$(R_p - R_f) / \sigma_p$	Rentabilidad ajustada a riesgo; > 1 es bueno, > 2 es excelente
Sortino Ratio	$(R_p - R_f) / \sigma_d$	Como el ratio Sharpe pero solo penaliza volatilidad negativa
Max Drawdown	$\max(D_t)$	Peor caída desde un máximo histórico
Win Rate	$W / (W + L)$	Porcentaje de operaciones ganadoras
Profit Factor	$\Sigma G / \Sigma L$	Ganancias brutas / pérdidas brutas; > 1.5 es deseable

2.5 Infraestructura de Despliegue: Docker y Microservicios

La arquitectura de microservicios, popularizada por empresas como Netflix y Amazon, consiste en descomponer una aplicación monolítica en servicios pequeños e independientes que se comunican entre sí a través de APIs o bases de datos compartidas (Newman, 2015).

El uso de Docker, lanzado como proyecto *open source* en 2013 y formalizado académicamente por Merkel (2014), se justifica principalmente por su portabilidad: al

empaquetar cada microservicio con sus dependencias, garantiza que el sistema se despliegue de forma idéntica en cualquier máquina Linux con Docker instalado. A su vez, el archivo `docker-compose.yml` opera como infraestructura-como-código (IaC), documentando la arquitectura general de forma ejecutable. Es importante precisar que la resiliencia operacional y la ejecución autónoma 24/7 no son intrínsecas a Docker por sí solo, sino que se delegan a Docker Compose mediante la política de recuperación `restart: always`, encargada de reiniciar los contenedores automáticamente ante fallos imprevistos.

TimescaleDB, la extensión de PostgreSQL para series temporales, fue seleccionada como motor de base de datos por su capacidad de particionar automáticamente los datos por tiempo (*hypertables*), optimizando las consultas de ventana temporal que son frecuentes en el análisis financiero (Timescale Inc., 2019).

*Nota de compatibilidad de despliegue: La infraestructura-como-código definida en el proyecto utiliza la sintaxis moderna de Docker Compose v2, eliminando la declaración inicial de versión del **schema** por estar deprecada. Por consiguiente, el despliegue de este sistema establece como requisitos mínimos operativos disponer de **Docker Engine 20.10+** y **Docker Compose Plugin v2.0+***

2.6 Framework Freqtrade y FreqAI

Freqtrade es un framework de trading algorítmico de código abierto escrito en Python con más de 34.000 estrellas en GitHub (mayo 2026). Sus principales características son:

- **Soporte multi-exchange:** Compatible con Binance, Kraken, OKX, entre otros, a través de la librería CCXT.
- **Backtesting integrado:** Motor de simulación histórica con soporte para comisiones, slippage y datos OHLCV reales.
- **FreqAI:** Módulo de Machine Learning que permite entrenar modelos (LightGBM, XGBoost, CatBoost, PyTorch) directamente integrados con la estrategia de trading, con re-entrenamiento automático mediante ventanas deslizantes.
- **Hyperopt:** Motor de optimización de hiperparámetros basado en algoritmos genéticos y Optuna.
- **Telegram Bot:** Interfaz de notificaciones y control remoto.
- **FreqUI:** Dashboard web para monitorización visual.

2.7 Trabajos Relacionados

Diversos trabajos académicos han explorado la combinación de Machine Learning y NLP para trading algorítmico. Jiang et al. (2021) combinaron LSTM con análisis de sentimiento de Twitter para predecir precios de Bitcoin, obteniendo un Sharpe Ratio de 0.85. Sin embargo, su sistema carecía de gestión de riesgo dinámica y operaba con un único activo.

Carta et al. (2021) propusieron Multi-DQN, un ensemble de agentes de aprendizaje por refuerzo profundo (Deep Q-Network) para la predicción y operativa en mercados de valores. Su arquitectura combina múltiples agentes DQN entrenados independientemente cuyas decisiones se agregan mediante votación, obteniendo mejores resultados que un agente único en términos de retorno acumulado. Su limitación principal radica en la ausencia de análisis de sentimiento externo y en la validación en un único régimen de mercado alcista, sin contemplar escenarios de crisis o lateralidad.

El presente TFG se diferencia de los trabajos anteriores en tres aspectos fundamentales: (1) la combinación de 7 capas de análisis independientes frente a los 1-2 señales de los sistemas anteriores, (2) la validación rigurosa en 4 escenarios de mercado distintos, y (3) la integración de análisis de sentimiento NLP en tiempo real como capa de filtrado, ausente en los sistemas basados puramente en aprendizaje por refuerzo como el de Carta et al. (2021).

Capítulo 3 - Análisis de Requisitos

3.1 Requisitos Funcionales

Tabla 5: Requisitos funcionales del sistema

ID	Requisito	Prioridad
RF-01	El sistema debe descargar datos de mercado (OHLCV) en tiempo real desde la API de Binance para 11 pares de criptomonedas	Alta
RF-02	El sistema debe calcular 18 indicadores técnicos multi-timeframe (RSI, StochRSI, MFI, MACD, BB Width, ATR, OBV, Log Returns, Return Std, features MTF H1) para cada par	Alta
RF-03	El sistema debe entrenar un modelo LightGBM Regressor con ventana deslizante de 30 días y re-entrenarse cada 2 horas	Alta
RF-04	El sistema debe predecir el porcentaje de cambio del precio en las próximas 20 velas (100 minutos en temporalidad de 5m)	Alta
RF-05	El sistema debe descargar titulares de noticias de 3 fuentes RSS (CoinDesk, CoinTelegraph, Bitcoin.com) cada 5 minutos	Alta
RF-06	El sistema debe analizar el sentimiento de los titulares con FinBERT, aplicar NER para identificar la moneda mencionada, y almacenar los resultados en TimescaleDB	Alta
RF-07	El sistema debe generar señales de entrada LONG cuando se cumplan simultáneamente 7 condiciones: régimen tendencial ($ADX > 20$), tendencia alcista (SMA200), zona de valor (EMA50), predicción IA positiva, sentimiento NLP no negativo, volumen superior a la media, y validación de Order Flow	Alta
RF-08	El sistema debe generar señales de entrada SHORT con las condiciones inversas a RF-07	Alta
RF-09	El sistema debe calcular un stop loss dinámico basado en $2 \times ATR$ para cada operación activa, con transición a trailing SAR cuando el beneficio supere el 2%	Alta
RF-10	El sistema debe bloquear nuevas operaciones si la pérdida acumulada del día supera el 10% (Circuit Breaker)	Alta
RF-11	El sistema debe dimensionar el tamaño de cada operación según la confianza de la IA (Kelly empírico al 40%), con descuento por volatilidad	Media
RF-12	El sistema debe consultar el Fear & Greed Index y bloquear LONGs en Extreme Greed (>85) y SHORTs en Extreme Fear (<15)	Media

RF-13	El sistema debe enviar notificaciones de apertura y cierre de operaciones vía Telegram	Media
RF-14	El sistema debe proporcionar una interfaz web (FreqUI) para la monitorización visual de operaciones y un dashboard Grafana para métricas históricas	Baja

3.2 Requisitos No Funcionales

Tabla 6: Requisitos no funcionales del sistema

ID	Requisito	Categoría
RNF-01	El sistema debe procesar cada ciclo de análisis (18 indicadores $\times$ 11 pares $\times$ 3 temporalidades) en menos de 30 segundos	Rendimiento
RNF-02	El sistema debe operar de forma autónoma 24/7 sin intervención humana	Disponibilidad
RNF-03	Las credenciales de API (Binance, Telegram) deben almacenarse en un archivo separado ('config_secrets.json') no incluido en el repositorio Git	Seguridad
RNF-04	El sistema debe poder desplegarse en cualquier servidor Linux con Docker instalado mediante un único script ('deploy_ubuntu.sh')	Portabilidad
RNF-05	El sistema debe consumir menos de 4 GB de RAM en estado estacionario	Eficiencia
RNF-06	El código debe seguir las convenciones de Python (PEP 8) y estar documentado con docstrings completas	Mantenibilidad
RNF-07	Los componentes críticos (Kelly, ATR, NLP) deben contar con tests unitarios automatizados (Pytest)	Fiabilidad
RNF-08	El sistema debe ser capaz de recuperarse automáticamente de caídas temporales de la API de Binance o los feeds RSS, con mecanismos de fallback	Tolerancia a fallos

3.3 Restricciones del Sistema

- 1. Exchange:** El sistema opera exclusivamente en Binance Futures (contrato perpetuo USDT-M).
- 2. Temporalidad base:** 5 minutos (M5) con análisis complementario en 15 minutos (M15) y 1 hora (H1).
- 3. Modo de operación:** Simulado (*dry-run*) durante el periodo del TFG. Migración a dinero real sujeta a los resultados del Forward-Testing.

4. **Apalancamiento:** $\times 10$ en modo aislado (*isolated margin*), lo que significa que cada operación solo puede perder el capital asignado a ella, no el balance total de la cuenta.
5. **Hardware mínimo:** 4 GB RAM, 2 CPUs, 20 GB disco (compatible con instancias gratuitas de Oracle Cloud o servidores de bajo coste).

3.4 Universo de Activos

El sistema monitoriza permanentemente los siguientes 11 pares de criptomonedas, seleccionados por su liquidez y capitalización de mercado:

Tabla 7: Universo de activos monitorizados

Par	Activo	Capitalización (Abr 2026)
BTC/USDT:USDT	Bitcoin	~\$1.3T
ETH/USDT:USDT	Ethereum	~\$380B
SOL/USDT:USDT	Solana	~\$65B
BNB/USDT:USDT	Binance Coin	~\$85B
ADA/USDT:USDT	Cardano	~\$18B
XRP/USDT:USDT	Ripple	~\$32B
DOT/USDT:USDT	Polkadot	~\$8B
LINK/USDT:USDT	Chainlink	~\$9B
AVAX/USDT:USDT	Avalanche	~\$12B
DOGE/USDT:USDT	Dogecoin	~\$22B
NEAR/USDT:USDT	NEAR Protocol	~\$5B

La selección prioriza activos con alto volumen diario de negociación ($> \$100M$) para minimizar el riesgo de deslizamiento (*slippage*) en la ejecución de órdenes.

Fuente: CoinGecko / CoinMarketCap. Datos consultados en abril de 2026. Las capitalizaciones de mercado son aproximadas y varían en tiempo real.

3.5 Metodología de Desarrollo

El desarrollo del sistema siguió una **metodología iterativa incremental** inspirada en los principios ágiles, adaptada al contexto individual de un TFG. El proceso se organizó en ciclos

de desarrollo de 2-4 semanas (sprints), donde cada iteración producía una versión funcional del sistema con incrementos progresivos de complejidad.

Las fases del ciclo iterativo fueron:

1. **Investigación y diseño:** Análisis de la literatura, selección de tecnologías y diseño de la arquitectura del sprint actual.
2. **Implementación:** Codificación de la funcionalidad planificada, siguiendo convenciones PEP 8 y documentación con docstrings.
3. **Testing:** Ejecución de tests unitarios, verificación de la integración con Docker y pruebas manuales en modo *dry-run*.
4. **Backtesting y evaluación:** Ejecución de backtests Walk-Forward para validar el impacto de los cambios en el rendimiento.
5. **Retrospectiva:** Análisis de resultados, identificación de problemas y planificación del siguiente sprint.

Este enfoque iterativo permitió evolucionar el sistema desde una versión mínima (v1.0, 5 features, target binario) hasta la versión final (v3.0, 18 features MTF, 7 capas de confluencia) en 6 iteraciones documentadas en el Capítulo 6.

Herramientas de gestión: Git (control de versiones), GitHub (repositorio remoto), Docker Compose (entorno reproducible), Makefile (automatización de tareas), Pytest (validación continua).

3.6 Planificación Temporal

El proyecto se desarrolló a lo largo de 4 meses (febrero – mayo 2026), con una dedicación estimada de **350 horas** distribuidas en las siguientes fases:

Tabla 8: Planificación temporal del proyecto

Fase	Periodo	Semanas	Horas	Entregable
F1 - Investigación y Estado del Arte	Feb 2026	2	40	Revisión bibliográfica, selección de frameworks
F2 - Diseño de la Arquitectura	Feb 2026	1	60	Docker Compose, esquema de BD, pipeline de datos
F3 - Implementación	Feb-Mar 2026	3	60	Bot funcional, 5 features, target binario, Hyperopt

v1.0 – v1.2				
F4 - Implementación v2.0 – v2.1	Mar 2026	1	40	12 features, regresión, ATR stop, filtro ADX
F5 - Implementación v3.0	Mar 2026	2	80	MTF features, NER per-coin, on-chain, MLOps
F6 - Backtesting y Validación	Abr 2026	3	35	Walk-Forward en 4 escenarios, análisis SHAP
F7 - Redacción de la Memoria	Abr – May 2026	4	35	Memoria completa, figuras, bibliografía
Total	Feb – May 2026	16	350	

3.7 Estimación de Costes

Se presenta una estimación de los costes asociados al desarrollo y operación del sistema, a efectos de evaluar la viabilidad económica del proyecto en un contexto profesional.

Tabla 9: Estimación de costes del proyecto

Concepto	Detalle	Coste unitario	Cantidad	Total
Personal				
Ingeniero Informático Junior	Desarrollo, testing, documentación	15 €/h	350 h	5.250 €
Hardware				
MacBook (amortización 4 años)	Equipo de desarrollo	1.500 € / 48 meses	4 meses	125 €
VPS (producción)	Oracle Cloud Free Tier / Hetzner CX22	0 – 5 €/mes	5 meses	0 – 25 €
Software				
Freqtrade, Docker, Grafana, TimescaleDB	Código abierto (licencia MIT/Apache)	0 €	-	0 €

FinBERT (HuggingFace)	Modelo pre- entrenado gratuito	0 €	-	0 €
API Binance (Futures Testnet)	Gratuita para modo simulado	0 €	-	0 €
Datos				
Datos de mercado (OHLCV via CCXT)	API pública de Binance	0 €	-	0 €
Feeds RSS (CoinDesk, CoinTelegraph)	Públicos y gratuitos	0 €	-	0 €
Fear & Greed API (alternative.me)	Pública y gratuita	0 €	-	0 €

El coste total del proyecto es de aproximadamente **5.375 €**, de los cuales el 97% corresponde al coste de personal. El uso exclusivo de herramientas de código abierto y APIs gratuitas reduce drásticamente los costes de software e infraestructura, demostrando la viabilidad de construir un sistema de trading de grado profesional con inversión mínima en licencias.

3.8 Análisis de Riesgos

Se identificaron los principales riesgos del proyecto y las estrategias de mitigación implementadas:

Tabla 10: Análisis de riesgos y mitigaciones

ID	Riesgo	Probabilidad	Impacto	Mitigación Implementada
R1	Caída de la API de Binance durante operación	Media	Alto	Freqtrade implementa reintentos automáticos con backoff; el bot pausa y reanuda sin pérdida de estado
R2	Fallo simultáneo de los 3 feeds RSS	Baja	Medio	‘FALLBACK_HEADLINES’ con 4 titulares de emergencia; fallback a sentimiento neutro (0.0)
R3	Overfitting del modelo LightGBM	Alta	Alto	Validación Walk-Forward (no hay filtración de datos futuros); re-entrenamiento cada 2h con ventana

				deslizante
R4	Pérdida de datos por fallo de TimescaleDB	Baja	Alto	Volumen Docker persistente; script 'backup_db.sh' con retención de 7 días
R5	Ejecución involuntaria con dinero real	Baja	Crítico	'dry_run: true' en configuración; exchange keys vacías; Circuit Breaker como última barrera
R6	Exposición de credenciales en Git	Media	Alto	'gitignore' para '.env' y 'config_secrets.json'; plantillas '*.example' en repositorio
R7	Incompatibilidad de dependencias Python	Media	Medio	Docker aísla cada servicio; versiones fijadas en 'Dockerfile'; parche de datasieve documentado
R8	Mercado lateral prolongado (whipsaw)	Alta	Medio	Filtro ADX > 20 (Capa 2) bloquea operaciones en mercados sin tendencia
R9	Exposición no autorizada del puerto de base de datos	Alta	Alto	Eliminación del binding de puerto en 'docker-compose.yml' y bloqueo explícito vía firewall (UFW)

Capítulo 4 - Diseño del Sistema

Este capítulo describe el modelo algorítmico del sistema: sus componentes matemáticos, la lógica de decisión multi-capa y los módulos de inteligencia artificial, procesamiento de lenguaje natural y gestión de riesgo. El objetivo es explicar qué hace el modelo y por qué, sin entrar en detalles de infraestructura de despliegue, que se tratan en el próximo capítulo.

4.1 Visión General del Modelo

El corazón del sistema es un motor de decisión de 7 capas de confluencia. La idea central es que ninguna señal individual es suficientemente fiable por sí sola: solo cuando múltiples filtros independientes apuntan en la misma dirección se emite una orden de trading. Esta arquitectura reduce drásticamente los falsos positivos y convierte al sistema en un preservador de capital por diseño.

Desde un punto de vista formal, el sistema puede modelarse como un sistema multimodal que integra tres fuentes de información heterogéneas:

- **Series temporales de precio (OHLCV):** procesadas mediante indicadores técnicos y un modelo de regresión supervisada (LightGBM).
- **Texto (titulares de noticias financieras):** procesado mediante un modelo de lenguaje pre-entrenado (FinBERT) con reconocimiento de entidades (NER).
- **Datos on-chain (Fear & Greed Index, BTC Dominance):** usados como señales de sentimiento macro del mercado.

Las tres fuentes convergen en el motor de decisión, que emite una señal de trading solo si se cumplen simultáneamente todas las condiciones de confluencia. La siguiente figura ilustra esta pirámide de decisión.

Motor de Decisión Multi-Capa

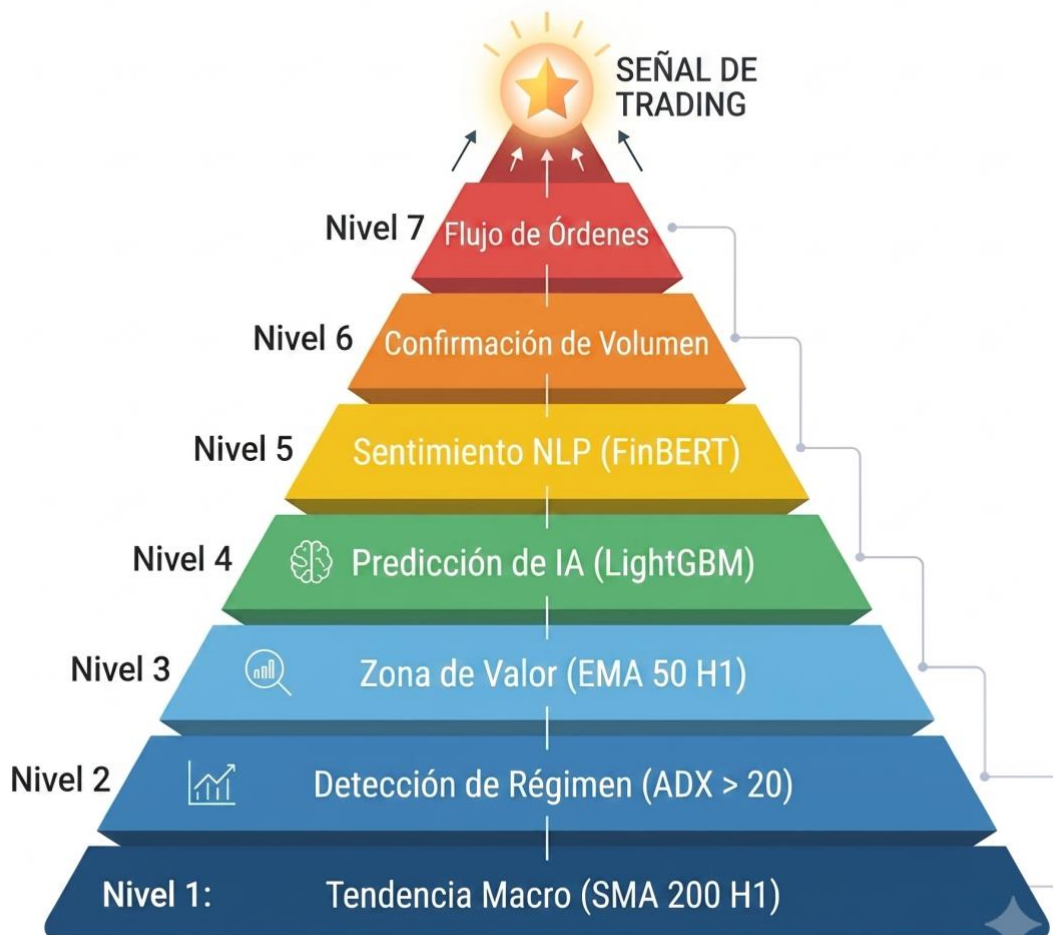


Ilustración 1: Motor de Decisión Multi-Capa - 7 capas de confluencia

4.2 Motor de Decisión Multi-Capa

El motor de decisión exige la confluencia de 7 capas para generar una señal de trading. La siguiente tabla describe las condiciones para señales LONG:

Tabla 11: Reglas de las 7 capas de confluencia (señal LONG)

Capa	Nombre	Condición LONG	Justificación
1	Macro H1	Precio > SMA 200 en H1	Solo operar a favor de la tendencia principal

2	Régimen ADX	ADX H1 > 20	No operar en mercados laterales (whipsaw)
3	Zona de Valor	Distancia al EMA50 H1 < 2.5%	No comprar lejos de la media (sobrecalentado)
4	Predicción IA	&s-price_change > umbral_long	LightGBM predice subida significativa
5	Sentimiento NLP	sentiment_score > -0.4	No comprar con noticias negativas
6	Volumen	Volumen > media 50 periodos	Confirmar interés real del mercado
7	Order Flow	Precio de entrada desde el libro de órdenes (use_order_book: true)	Reduce el slippage en la ejecución

Para señales SHORT, las condiciones son las inversas de las capas 1, 3, 4 y 5. Las capas 2, 6 y 7 operan de forma simétrica.

4.2.1 Ciclo de Vida de una Operación

La Figura 4.4 ilustra el ciclo de vida completo de una operación de trading, desde la recepción de una nueva vela de precio hasta la ejecución o rechazo de la orden en el exchange. Cada vela de 5 minutos dispara la ejecución secuencial de las 7 capas de análisis; solo si todas las condiciones se cumplen simultáneamente, la operación pasa al gate de seguridad (confirm_trade_entry) donde el Circuit Breaker, el filtro Fear & Greed y el control de Portfolio Heat realizan la validación final.

Ciclo de Vida de una Operación

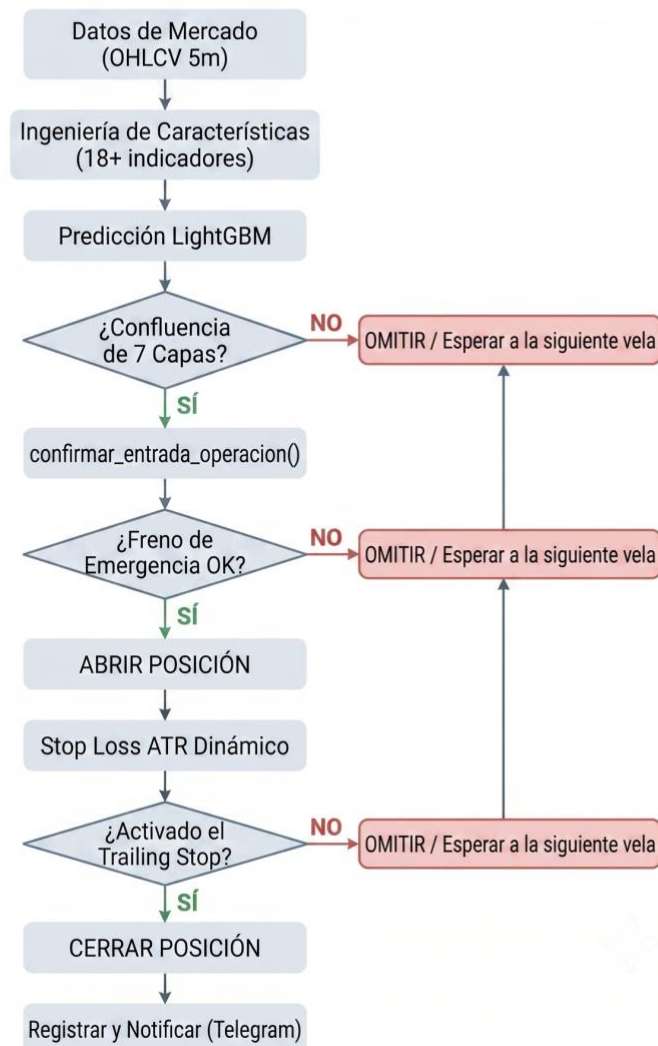


Ilustración 2: Ciclo de vida de una operación de trading

4.3 Componente de Inteligencia Artificial (LightGBM)

4.3.1 Feature Engineering: De 5 a 18 Features

La ingeniería de características es el componente más crítico del modelo predictivo. A lo largo de las iteraciones del sistema (v1.0 → v3.0), el pipeline de features se amplió de 5 indicadores básicos a 18 features multi-dimensionales organizadas en 7 categorías.

Tabla 12: Features del modelo organizadas por categoría

	Feature	Rol	Categoría	Fórmula / Descripción	Justificación
1	Rsi	Feature ML	Momentum	RSI(14)	Sobrecompra/sobreventa (Wilder, 1978)
2	stoch_rsi	Feature ML	Momentum	StochRSI(14,14,3,3)	RSI del RSI: más sensible a cambios rápidos
3	Mfi	Feature ML	Momentum	MFI(14)	RSI ponderado por volumen real
4	macd_hist	Feature ML	Momentum	MACD(12,26,9) Histograma	Cruces de tendencia y divergencias
5	bb_width	Feature ML	Volatilidad	(BB_upper - BB_lower) / BB_middle	Amplitud normalizada de Bandas de Bollinger
6	atr_norm	Feature ML	Volatilidad	ATR(14) / close	Volatilidad relativa al precio
7	obv_norm	Feature ML	Volumen	OBV / SMA(OBV, 50)	Presión compradora/vendedora normalizada
8	log_return	Feature ML	Estadístico	$\ln(\text{close} / \text{close}[-1])$	Distribución más gaussiana para ML
9	return_std	Feature ML	Estadístico	std(returns, 20)	Régimen de volatilidad reciente
10	candle_dir	Feature ML	Precio	close / open	Dirección y fuerza de la vela
11	pct_change	Feature ML	Precio	$(\text{close} - \text{close}[-1]) / \text{close}[-1]$	Cambio porcentual base
12	sentiment_score	Filtro de señal*	Fundamental	NLP per-coin via TimescaleDB	Cuantifica el sentimiento de mercado para validar la entrada
13	price_ratio_5m_1h	Feature ML	MTF	close_5m / close_1h	Divergencia micro vs macro
14	dist_sma200_1h	Feature ML	MTF	$(\text{close} - \text{SMA200_1h}) /$	Posición relativa a tendencia macro

				SMA200_1h	
15	dist_ema50_1h	Feature ML	MTF	(close - EMA50_1h) / EMA50_1h	Zona de valor en timeframe superior
16	adx_1h_norm	Feature ML	MTF	ADX_1h / 100	Fuerza de tendencia macro normalizada
17	hour_sin/cos	Feature ML	Temporal	sin/cos($2\pi \times \text{hora} / 24$)	Codificación cíclica horaria
18	day_sin/cos	Feature ML	Temporal	sin/cos($2\pi \times \text{día} / 7$)	Codificación cíclica semanal

Nota técnica: Las variables categorizadas como “Filtro de señal” son consultadas estructuralmente en la función ‘populate_entry_trend()’ para condicionar y validar entradas al mercado, pero no son inyectadas como input al modelo LightGBM durante el entrenamiento. Esta disociación arquitectónica explica por qué tener una varianza de 0.0 durante el backtesting histórico (debido al modo fallback NLP) no compromete la integridad algorítmica ni el proceso de reducción de dimensionalidad.

4.3.2 Codificación Temporal Cíclica

Un aspecto técnico relevante es la codificación de variables temporales. Representar la hora como un entero (0–23) induce al modelo a interpretar que las 23:00h y las 00:00h están “lejos” cuando en realidad son consecutivas. La codificación seno/coseno preserva la circularidad temporal:

```
hour_sin = sin(2π × hour / 24)
hour_cos = cos(2π × hour / 24)
```

4.3.3 Target de Regresión y Horizonte Temporal

El modelo predice el porcentaje de cambio del precio en las próximas N velas:

```
# En set_freqai_targets():
df["&s-price_change"] = (
    df["close"].shift(-
self.freqai_info["feature_parameters"]["label_period_candles"])
    / df["close"] - 1
```

)

Con `label_period_candles = 20` (100 minutos en 5m), el modelo aprende a predecir movimientos de precio en un horizonte temporal que equilibra la señal (suficiente tiempo para un movimiento significativo) con el ruido (no tan lejos como para perder poder predictivo).

4.3.4 Re-entrenamiento Continuo (Walk-Forward)

El framework FreqAI implementa automáticamente el esquema Walk-Forward: el modelo se entrena con una ventana deslizante de 30 días de datos históricos y se re-entrena cada 2 horas. Este mecanismo mitiga el Concept Drift al reducir la latencia de adaptación a nuevos regímenes de mercado.

4.4 Componente NLP (FinBERT + NER)

4.4.1 Pipeline de Procesamiento

El pipeline de Procesamiento de Lenguaje Natural se ejecuta en un contenedor independiente:

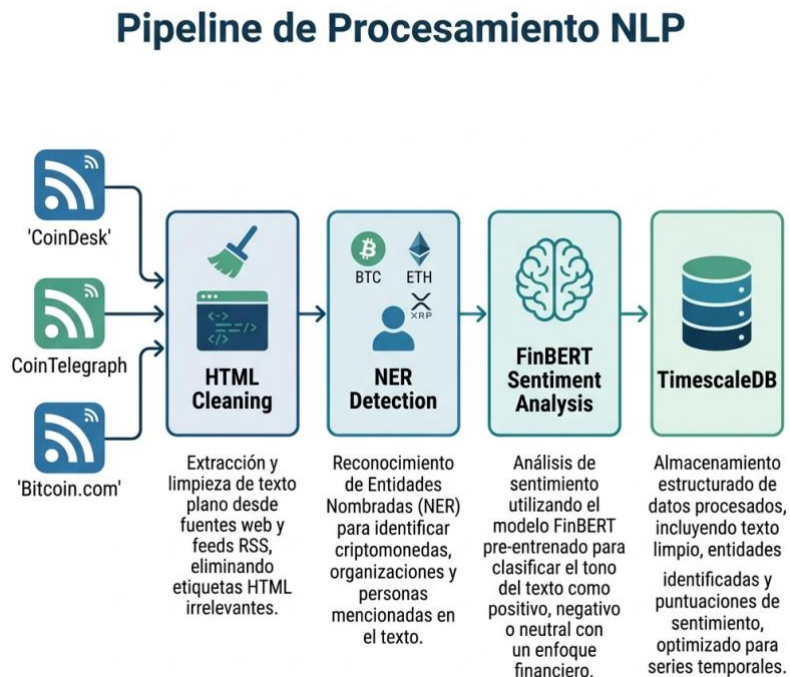


Figura 4.3: Pipeline de Procesamiento NLP

1. **Ingesta (RSS):** Descarga titulares de CoinDesk, CoinTelegraph y Bitcoin.com usando `feedparser` cada 5 minutos.

2. **Limpieza HTML:** Elimina etiquetas HTML residuales con `BeautifulSoup` para evitar tokens espurios en la inferencia.
3. **NER (Named Entity Recognition):** Utiliza un diccionario de 50+ alias (`COIN_ALIASES`) para detectar qué criptomoneda se menciona. Ejemplo: “Vitalik anuncia actualización” → ETH.
4. **Inferencia FinBERT:** Envía el titular limpio al modelo `ProsusAI/finbert` de HuggingFace, obteniendo un score en $[-1, +1]$.
5. **Almacenamiento:** Inserta el resultado en `coin_sentiment` de TimescaleDB.
6. **Fallback:** Si no hay datos per-coin para un activo, el bot utiliza el sentimiento global promedio.

4.4.2 NLP como Filtro, no como Generador

Un hallazgo relevante del desarrollo es que el componente NLP aporta más valor como mecanismo de protección (filtrar operaciones malas) que como generador de señales (encontrar operaciones buenas). La Capa 5 del motor de decisión bloquea entradas LONG cuando el sentimiento per-coin es muy negativo ($\text{score} \leq -0.4$), evitando pérdidas en cascada durante crisis mediáticas.

Nota: Durante el backtesting histórico, la Capa 5 operó en modo fallback (score = 0.0 constante) porque los titulares RSS históricos no estaban disponibles. La validación empírica completa del componente NLP se realizará durante el período de Forward-Testing actualmente en curso

4.4.3 Fallback y Resiliencia

Si no se encuentran datos de sentimiento recientes en TimescaleDB, el sistema asigna automáticamente una puntuación neutra ($\text{score} = 0.0$). Para datos desactualizados (stale data) de más de 15 minutos, se aplica la misma lógica de fallback.

4.5 Gestión de Riesgo Cuantitativa

El sistema implementa 4 capas de protección patrimonial independientes que actúan de forma coordinada:



Ilustración 3: Capas de Gestión de Riesgo del sistema

4.5.1 Kelly empírico al 40% ('custom_stake_amount')

El sistema implementa una variante simplificada del Criterio de Kelly denominada **Kelly empírico al 40%**: en lugar de calcular dinámicamente f^* a partir de la probabilidad de ganancia y el ratio de pago estimados por el modelo (lo que requeriría calibración continua y es susceptible a overfitting), se fija un parámetro base $k = 0.40$ que actúa como cota superior de la fracción de capital a arriesgar. Este valor fue determinado empíricamente durante el proceso de Hyperopt para maximizar el rendimiento ajustado a riesgo. Sobre este parámetro base se aplican dos factores de ajuste dinámico: la confianza de la predicción IA (`confidence = abs(last_prediction)`) y un descuento por volatilidad (`volatility_discount = 1 - min(atr_norm, 0.5)`). Esta implementación preserva la intuición central del Criterio de

Kelly, asignar más capital cuando la confianza es alta y el mercado es tranquilo, sin asumir la estimación precisa de probabilidades que requiere la fórmula original.

```
def custom_stake_amount(self, ...):
    # Kelly empírico al 40%
    kelly = 0.40

    # Ajuste por volatilidad (VaR implícito)
    volatility_discount = 1 - min(atr_norm, 0.5)

    # Ajuste por confianza de la IA
    confidence = abs(last_prediction)

    stake = balance * kelly * confidence * volatility_discount

    return max(min(stake, balance * 0.4), min_stake)
```

El sistema invierte más capital cuando la IA tiene alta confianza y el mercado es poco volátil, y reduce la exposición en condiciones de incertidumbre.

4.5.2 Stop Loss Dinámico Basado en ATR

La fórmula implementada es: $\text{stop} = -(2 \times \text{ATR}_{14}) / \text{precio_actual}$. Los límites de seguridad se aplican mediante: $\max(\min(\text{stop}, -0.005), -0.03)$. La función $\min(\dots, -0.005)$ asegura que el stop no sea más ajustado que el -0.5% (piso mínimo), mientras que $\max(\dots, -0.03)$ garantiza que no sea más amplio que el -3.0% (techo máximo de pérdida permitida).

4.5.3 Trailing Stop Híbrido

El sistema implementa un stop loss de dos fases:

- **Fase 1 (Protección ATR):** Mientras la operación no ha superado el 2% de beneficio, el stop loss se calcula como $-(2 \times \text{ATR}_{14}) / \text{precio}$. Esto permite que el precio “respire” según la volatilidad real del momento, evitando salidas prematuras por ruido de mercado.
- **Fase 2 (Trailing SAR):** Una vez que el beneficio supera el 2%, el stop se ciñe al valor del Parabolic SAR, capturando la tendencia de forma agresiva y forzando la salida cuando la inercia macro muere.

4.5.4 Circuit Breaker (‘confirm_trade_entry’)

Antes de confirmar cualquier operación, el sistema verifica que las pérdidas acumuladas del día no superen el 10%:

```
def confirm_trade_entry(self, ...):
    daily_profit = sum(trade.profit for trade in open_trades_today)
```

```
if daily_profit < -0.10:
    logger.warning("CIRCUIT BREAKER ACTIVADO: pérdida diaria > 10%")
    return False
return True
```

4.5.5 Filtro On-Chain (Fear & Greed)

El último filtro consulta el Fear & Greed Index almacenado en TimescaleDB y bloquea operaciones en extremos emocionales del mercado, donde las reversiones estadísticas son más probables. Bloquea LONGs si Fear & Greed > 85 (mercado sobrecalentado) y bloquea SHORTs si Fear & Greed < 15 (posible rebote inminente).

4.6 Pipeline On-Chain

El servicio onchain_data descarga datos macroeconómicos cada 15 minutos: Fear & Greed Index (desde api.alternative.me, valor entre 0 y 100), BTC Dominance y Market Cap (desde api.coingecko.com) para contexto macro del mercado. Estos datos se almacenan en la tabla market_data de TimescaleDB y se consultan en confirm_trade_entry() como última barrera de protección.

Capítulo 5 - Solución Propuesta: Bot y Aplicación

Este capítulo describe la implementación técnica del sistema: la arquitectura de microservicios, la infraestructura Docker, los pipelines de datos y los mecanismos de monitorización y despliegue. En este capítulo abordaremos el problema de convertir el modelo descrito en el Capítulo 4 en un sistema operativo con propiedades de disponibilidad, eficiencia y tolerancia a fallos.

5.1 Arquitectura General de Microservicios

El sistema sigue una arquitectura de microservicios orquestada con Docker Compose. Cada servicio se ejecuta en un contenedor independiente con su propio sistema de archivos, dependencias y ciclo de vida, comunicándose exclusivamente a través de la base de datos compartida TimescaleDB.

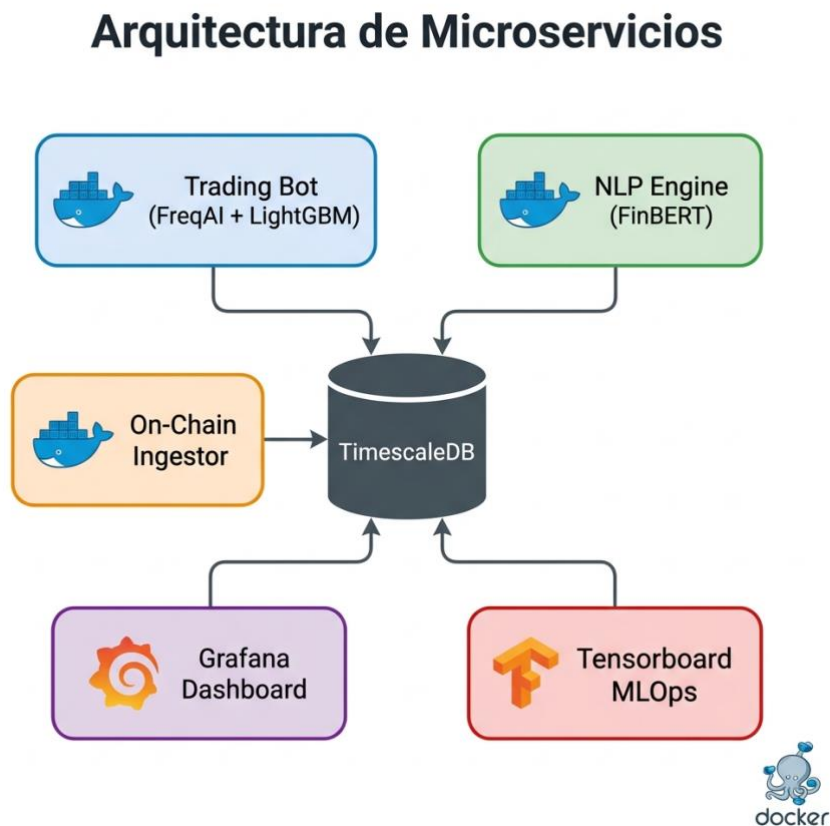


Ilustración 4: Arquitectura de microservicios del sistema

Tabla 13: Servicios del sistema y configuración de red

	Servicio	Contenedor	Puerto	Imagen Base	Función
1	freqtrade	freqtrade_elite_bot	8081	freqtradeorg/freqtrade:stable_freqai	Bot principal + FreqAI + LightGBM
2	sentiment_analysis	sentiment_engine	-	Custom (Dockerfile.sentiment)	Motor NLP FinBERT + NER
3	Timescaledb	freqtrade_db	5432	timescale/timescaledb:latest-pg14	Base de datos de series temporales
4	grafana	freqtrade_viz	3000	grafana/Grafana	Dashboard de visualización
5	onchain_data	onchain_engine	-	Custom (Dockerfile.onchain)	Ingestor Fear & Greed / BTC Dominance
6	tensorboard	ml_ops_tensorboard	6006	tensorflow/tensorflow:2.15.0	Monitorización MLOps

Nota técnica: Se fija la versión 2.15.0 para asegurar la reproducibilidad del entorno. Se emplea la directiva 'platform: linux/amd64' para garantizar la compatibilidad en arquitecturas ARM (ej. Apple Silicon), dado que la imagen oficial de TensorFlow no ofrece variante nativa ARM para esta versión.

5.2 Flujo de Datos entre Servicios

El flujo de datos del sistema sigue un patrón de productor-consumidor asíncrono:

1. Productores (escriben en TimescaleDB):

- `sentiment_engine`: Escribe en la tabla `coin_sentiment` cada 5 minutos.
- `onchain_engine`: Escribe en la tabla `market_data` cada 15 minutos.

2. Consumidor (lee de TimescaleDB):

- `freqtrade_elite_bot`: Lee el sentimiento per-coin y el Fear & Greed Index antes de cada vela de 5 minutos. La consulta se ejecuta dentro de `populate_indicators()`, garantizando que los datos de sentimiento están disponibles antes de que el modelo LightGBM realice su predicción.

3. Almacenamiento (TimescaleDB):

- Recibe datos de 3 productores (bot, NLP, on-chain).
- Los almacena en hypertables particionadas por tiempo.
- Sirve como bus de datos central del sistema.

5.2.1 Mecanismos de Sincronización y Coherencia de Datos

La arquitectura de microservicios introduce un problema de temporización entre los contenedores `sentiment_engine` y `freqtrade_bot`. Aunque ambos dependen de TimescaleDB mediante la directiva `depends_on`, no existe sincronización entre ellos. Esto puede dar lugar a tres situaciones problemáticas: la lectura de datos de sentimiento desfasados (*stale data*), condiciones de carrera en los *timestamps* de escritura y lectura, y el problema del arranque en frío (*cold start*), en el que el bot puede comenzar su primer ciclo de evaluación antes de que el motor NLP haya completado su primera ingesta.

El sistema mitiga este riesgo mediante un mecanismo de *fallback*: si la consulta SQL no encuentra datos de sentimiento recientes, asigna automáticamente una puntuación neutra (`score = 0.0`). Esto evita errores en tiempo de ejecución, pero no elimina el posible desfase temporal. Como mejora futura, se proponen dos soluciones complementarias.

A nivel de infraestructura, se puede añadir un *healthcheck* en el `docker-compose.yml` del `sentiment_engine` que impida al bot arrancar hasta que exista al menos una inserción válida en la base de datos:

```
sentiment_engine:
  healthcheck:
    test: ["CMD-SHELL", "psql -U admin -d tfg -c \"SELECT 1 FROM coin_sentiment
WHERE timestamp > NOW() - INTERVAL '10 minutes' LIMIT 1;\" | grep -q 1"]
    interval: 30s
    retries: 5
```

A nivel de aplicación, la estrategia puede incorporar una comprobación explícita que descarte datos caducados:

```
# Validación estricta de latencia de datos NLP
if last_nlp_timestamp < datetime.now(timezone.utc) - timedelta(minutes=15):
    current_sentiment_score = 0.0 # Forzar fallback por stale data
```

La implementación de ambas mejoras garantizaría que el sistema nunca tome decisiones de trading basadas en datos de sentimiento desactualizados.

5.2.2 Política de Reinicio y Tolerancia a Fallos

Todos los servicios están configurados con `restart: always` (excepto Tensorboard con `unless-stopped`). Si un servicio falla:

- TimescaleDB tiene healthcheck configurado (`pg_isready`) con 5 reintentos.
- El bot principal depende de TimescaleDB (`depends_on`), garantizando que la base de datos esté lista antes de arrancar.
- Los servicios NLP y On-Chain implementan bloques `try/except` con logging de errores, permitiendo que continúen operando ante fallos puntuales de los feeds RSS o APIs externas.

5.3 Entorno de Desarrollo

Tabla 14: Stack tecnológico del entorno de desarrollo

Componente	Tecnología	Versión	Justificación
Lenguaje principal	Python	3.13	Estándar en ML y finanzas cuantitativas
Framework de trading	Freqtrade	stable_freqai	Framework OSS más maduro para cripto
Motor ML	LightGBM	4.x	Superior en datos tabulares (Grinsztajn, 2022)
NLP	Transformers (HuggingFace)	4.30	Acceso a FinBERT pre-entrenado
Base de datos	TimescaleDB	14 (PG14)	Hypertables optimizadas para time-series
Orquestación	Docker Compose	v3	Despliegue reproducible multi-servicio
Visualización	Grafana	latest	Dashboard profesional con alertas
MLOps	Tensorboard	latest	Estándar de la industria para tracking ML
Tests	Pytest	8.x	Framework de testing estándar en Python
Notificaciones	Telegram Bot API	-	Alertas en tiempo real al operador

5.4 Implementación de los Componentes Principales

5.4.1 Señales de Entrada ('populate_entry_trend')

La generación de señales requiere la confluencia de las 7 capas. En pseudocódigo:


```

SEÑAL LONG = (
    precio > SMA_200_H1                [Capa 1: Macro]
    AND ADX_H1 > 20                     [Capa 2: Régimen]
    AND dist_EMA50_H1 < 2.5%           [Capa 3: Zona de Valor]
    AND predicción_IA > umbral_long    [Capa 4: ML]
    AND sentimiento_NLP > -0.4         [Capa 5: NLP]
    AND volumen > media_50_periodos    [Capa 6: Volumen]
    AND order_book_imbalance > 0       [Capa 7: Order Flow]
)

```

5.4.2 Señales de Salida

Las salidas se gestionan mediante 4 mecanismos complementarios:

1. **Trailing Stop Positivo:** Se activa cuando el beneficio supera el 3% (offset), fijando un trailing del 1%.
2. **Stop Loss Dinámico ATR:** Calculado por `custom_stoploss()` en cada tick.
3. **ROI Mínimo:** Tabla de ROI progresiva definida en `config.json`.
4. **Trailing SAR:** Cuando el beneficio supera el 2%, el stop transiciona al Parabolic SAR para capturar tendencias fuertes.

5.5 Motor NLP ('sentiment_ingestor.py')

El motor NLP es un microservicio independiente de 200+ líneas que ejecuta un ciclo infinito cada 5 minutos:

1. Descarga titulares RSS con `feedparser`.
2. Limpia HTML con `BeautifulSoup(text, "html.parser").get_text(separator=" ")`.
3. Detecta entidades con regex sobre `COIN_ALIASES` (50+ alias: "Vitalik" → ETH, "Ripple" → XRP, etc.).
4. Clasifica sentimiento con `ProsusAI/finbert` via `transformers.pipeline("sentiment-analysis")`.
5. Almacena el resultado en TimescaleDB mediante SQLAlchemy: el engine creado con `create_engine()` se usa como destino de `df.to_sql()` para inserciones masivas o mediante `session.execute()` para inserciones individuales, persistiendo el score de sentimiento junto con el par y el timestamp.

El separador " " en `get_text()` fue un bug descubierto durante el desarrollo: sin él, "Bitcoin rallies 10%Breaking news" se concatenaba sin espacio, confundiendo al tokenizador de FinBERT.

5.6 Motor On-Chain ('onchain_ingestor.py')

Microservicio que descarga datos macro cada 15 minutos:

- **Fear & Greed Index:** API `api.alternative.me/fng/?limit=1`
- **BTC Dominance:** API `api.coingecko.com/api/v3/global`

Almacena en la tabla `market_data` de TimescaleDB. Implementa reintentos con backoff exponencial ante fallos de red.

5.7 Dockerización y Orquestación

5.7.1 Dockerfile Principal

El Dockerfile del bot principal parte de la imagen oficial `freqtradeorg/freqtrade:stable_freqai` e instala las dependencias adicionales:

- `sqlalchemy`, `psycopg2-binary`, `lightgbm`, `scikit-learn`, `pandas`, `xgboost`, `tensorboard`, donde `sqlalchemy` y `psycopg2-binary` son imprescindibles para la conexión a TimescaleDB.
- Dependencias de sistema: `gcc`, `libpq-dev` (`apt-get`), necesarias para compilar el driver PostgreSQL nativo
- `torch>=2.6.0` con índice CPU-only (evitando arrastrar CUDA, ahorrando cerca de 2GB de imagen)
- Parche de `datasieve 0.1.9` en build-time para corregir un bug de atributo

5.7.2 Gestión de Secretos

Las credenciales sensibles (API keys de Binance, token de Telegram) se almacenan en dos archivos excluidos del control de versiones:

- `.env`: Variables de entorno para Docker Compose (contraseñas de BD, Grafana).
- `config_secrets.json`: Configuración de Freqtrade con API keys del exchange y Telegram.

Ambos archivos están en `.gitignore` y se proporcionan plantillas (`*.example`) para facilitar la configuración.

5.8 Pipeline MLOps

El sistema implementa logging interno de la actividad de la IA para trazabilidad:

- **log_prediction_metrics():** Registra cada predicción del modelo (par, señal, confianza, sentimiento, ADX).
- **Logging de señales:** En `populate_entry_trend`, registra cada señal generada con sus condiciones.
- **Logging de confirmación:** En `confirm_trade_entry`, registra si una operación fue confirmada o rechazada por el Circuit Breaker.
- **Tensorboard:** El servicio 6 expone los modelos en el puerto 6006 para visualización de métricas de entrenamiento.

5.9 Diseño de la Base de Datos (TimescaleDB)

TimescaleDB se configura con dos tablas principales (hypertables particionadas por tiempo):

Tabla coin_sentiment: Almacena el sentimiento per-coin de cada titular analizado.

- `timestamp` (TIMESTAMPTZ): Fecha y hora del análisis
- `coin` (VARCHAR): Ticker de la criptomoneda detectada por NER
- `sentiment` (FLOAT): Score [-1.0, +1.0]
- `source` (VARCHAR): Fuente RSS del titular
- `headline` (TEXT): Titular original limpio

Tabla market_data: Almacena datos macroeconómicos on-chain.

- `timestamp` (TIMESTAMPTZ): Fecha y hora de la descarga
- `fear_greed` (INTEGER): Índice [0, 100]
- `btc_dominance` (FLOAT): Dominancia de Bitcoin [0, 100]%
- `total_market_cap` (FLOAT): Capitalización total en USD

Capítulo 6 - Evaluación del Modelo de Trading

6.1 Metodología Experimental: Walk-Forward Analysis

La validación del sistema se realizó mediante **Walk-Forward Analysis**, el estándar de la industria para evaluar estrategias de trading algorítmico. A diferencia del backtesting tradicional (train en todo el dataset, test en el mismo dataset), el Walk-Forward divide los datos en ventanas secuenciales donde el modelo se entrena con datos pasados y se evalúa con datos futuros nunca vistos.

La configuración de FreqAI implementa este esquema automáticamente:

- **Ventana de entrenamiento:** 30 días de datos históricos (sliding window).
- **Frecuencia de re-entrenamiento:** Cada 2 horas (24 velas de 5 minutos).
- **Periodo de evaluación:** Los datos posteriores al entrenamiento (out-of-sample).

Esta metodología previene el *lookahead bias* (filtración de datos del futuro) y proporciona métricas representativas del rendimiento esperado en producción real.

Consideración sobre la separación entre optimización y evaluación: Los parámetros de la estrategia (umbrales de la IA, parámetros ADX, Kelly empírico) se optimizaron mediante Hyperopt utilizando periodos históricos. Aunque la metodología Walk-Forward del modelo LightGBM garantiza que el modelo predictivo nunca ve datos futuros durante el entrenamiento, los parámetros fijos de la estrategia (umbrales, filtros) se calibraron con exposición a los mismos periodos de mercado que aparecen en los resultados del backtesting. Esta limitación es inherente al uso de Hyperopt sin un periodo de test completamente reservado (hold-out set), y constituye una fuente de optimismo potencial en las métricas reportadas. Los resultados del Forward-Testing actualmente en curso proporcionarán una evaluación verdaderamente out-of-sample de estos parámetros.

Nota sobre la capa NLP en backtesting: El motor de análisis de sentimiento FinBERT opera en tiempo real procesando titulares RSS almacenados en TimescaleDB. Durante el backtesting histórico, los titulares correspondientes a las fechas analizadas no están disponibles en la base de datos, por lo que el sistema activa el mecanismo de fallback configurado (sentimiento neutro, score = 0.0) durante la práctica totalidad de las operaciones simuladas. En consecuencia, los resultados del backtesting reportados en las secciones 6.3.1 a 6.3.4 deben

interpretarse como resultados del sistema con la Capa 5 (Sentimiento NLP) en modo fallback, no como resultado del sistema completo de 7 capas con NLP activo. La validación del componente NLP se realizará durante el periodo de Forward-Testing actualmente en curso.

6.1.1 Limitaciones del Motor de Backtesting

Una limitación inherente al motor de *backtesting* de Freqtrade es la omisión del *funding rate* (tasa de financiación) en contratos perpetuos. El *funding rate* es un mecanismo de pagos periódicos entre posiciones largas y cortas, estructurado para anclar el precio del derivado al activo subyacente. Dado que el sistema opera con apalancamiento $\times 10$ y las operaciones tienen una duración media de 3 a 8 horas, el impacto de estas primas sobre el balance se amplifica sustancialmente.

La exclusión de este factor induce sesgos estructurales en la simulación. En escenarios alcistas (*Bull Market*) y laterales (*Lateral*), la tasa de financiación suele ser positiva (estimada históricamente en 0.01% por cada 8 horas, pagada típicamente por las posiciones largas). Al no registrarse este coste operativo, las métricas de rendimiento reportadas presentan un evidente sesgo de optimismo. Por el contrario, durante escenarios de caída sostenida (*Crash* y *Bear Market*), las tasas tienden a volverse negativas (estimadas en -0.005% cada 8 horas, pagado por los *shorts* a los *longs*). En este contexto, el sistema dejaría de percibir ingresos adicionales al operar en corto, introduciendo un sesgo de conservadurismo en los resultados.

Para asegurar una cuantificación exacta de la esperanza matemática del sistema, se propone como trabajo futuro la extracción e integración de datos históricos de *funding rate* desde la API de Binance. Este perfeccionamiento técnico permitirá calibrar retrospectivamente el simulador y ajustar el rendimiento neto de la estrategia.

6.2 Escenarios de Mercado

Nota sobre el cálculo de rendimientos: Todos los porcentajes de rendimiento reportados en las secciones 6.3.1 a 6.3.4 se expresan sobre el capital total de la cuenta (balance efectivo), no sobre el capital nocional apalancado. El apalancamiento $\times 10$ en isolated margin significa que cada operación individual utiliza como margen una fracción del balance, limitando la pérdida máxima de cada trade al capital asignado a esa posición. Freqtrade gestiona automáticamente la conversión entre retorno sobre margen y retorno sobre balance en su motor de backtesting para contratos de futuros USDT-M.

El mercado fue dividido rigurosamente en 4 regímenes para evaluar la robustez del sistema ante condiciones heterogéneas:

Tabla 15: Tabla comparativa de los periodos seleccionados.

Escenario	Periodo	Comportamiento del Mercado
Bull Market	Abril – Mayo 2025	Tendencia alcista sólida (+11.74%)
Bear Market	Julio – Octubre 2025	Tendencia bajista sostenida (-36%)
Lateral (Whipsaw)	Enero – Marzo 2025	Movimiento errático sin dirección (+13.49% con muchos retrocesos)
Crash	Octubre – Diciembre 2025	Caída extrema (-34.57%)

Nota metodológica sobre el benchmark: Los valores de Buy & Hold reportados corresponden al rendimiento promedio equiponderado de los 11 pares de las criptomonedas monitorizadas (Tabla 7) durante cada periodo de evaluación, calculado asumiendo una posición LONG abierta al inicio del periodo y cerrada al final sin gestión activa. Los datos de precio se obtuvieron de la API de Binance Futures mediante el mismo pipeline de descarga OHLCV utilizado por el sistema. Los escenarios Bear Market y Crash son periodos contiguos sin solapamiento de datos; las fechas exactas de inicio y fin de cada escenario están definidas en la configuración de backtesting del repositorio.

6.3 Resultados por Escenario

6.3.1 Escenario 1: Bull Market (Abril – Mayo 2025)

Rendimiento Bot vs Buy & Hold por Escenario

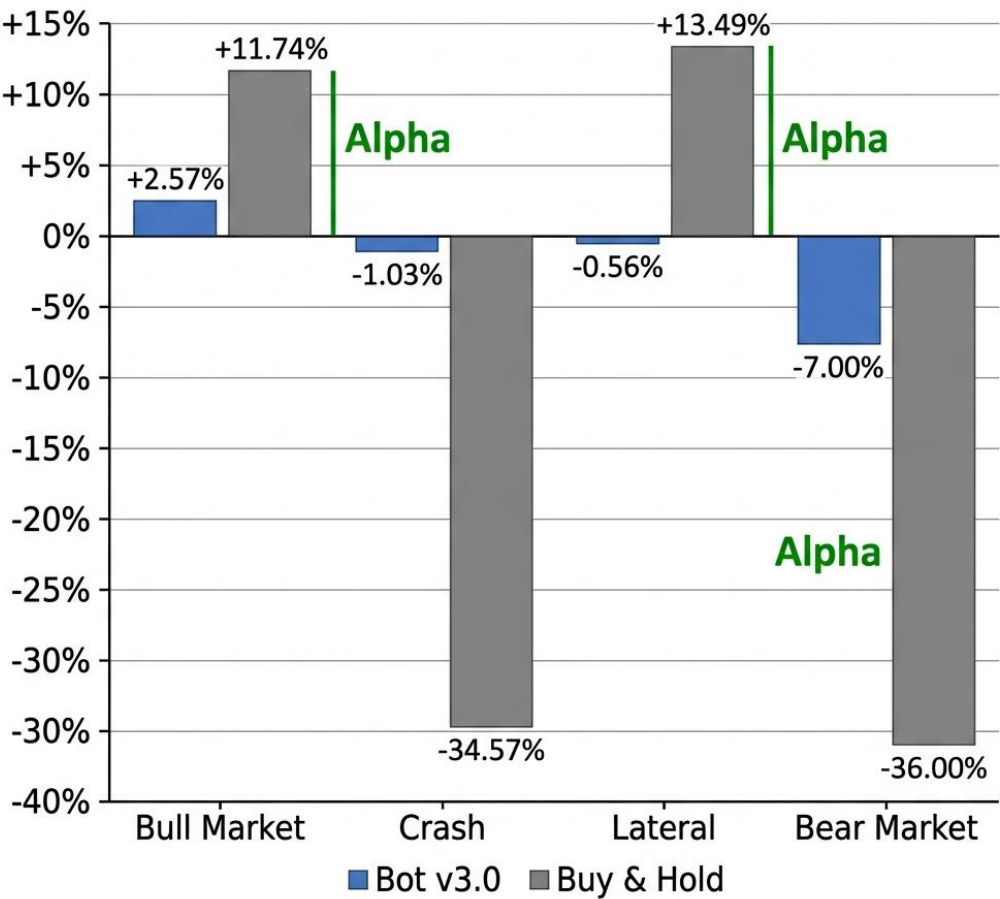


Ilustración 5: Rendimiento Bot vs Buy & Hold por escenario

Tabla 16: Resultados del backtesting - Bull Market

Métrica	Valor
Trades	15
Rendimiento neto	+2.57%
Win Rate	53.3%
Sharpe Ratio	0.06
Sortino Ratio	0.13
Max Drawdown	2.97%

Profit Factor	1.03
Benchmark (Buy & Hold)	+11.74%
Alpha generado	-9.17%

Análisis: El sistema obtiene un rendimiento neto del +2.57% en un mercado alcista fuerte (+11.74%), generando un alpha negativo de -9.17 puntos porcentuales frente al benchmark. Este resultado refleja la naturaleza conservadora del motor de decisión de 7 capas: la exigencia de confluencia simultánea de todos los filtros genera pocas señales (15 trades en 2 meses), priorizando la calidad sobre la cantidad. Con solo 15 operaciones, el sistema captura una fracción limitada del movimiento alcista. El Win Rate del 53.3% y el Profit Factor de 1.03 confirman una rentabilidad marginal, mientras que el Max Drawdown del 2.97% demuestra un control de riesgo efectivo. Es importante señalar que en mercados alcistas fuertes, la estrategia pasiva siempre superará a un sistema conservador con gestión de riesgo activa; sin embargo, este es el precio que se paga por la protección asimétrica que el sistema ofrece en escenarios adversos. El número limitado de operaciones sugiere que el filtro de confluencia de 7 capas puede ser excesivamente restrictivo en tendencias alcistas sostenidas, lo cual constituye una área de mejora futura.

6.3.2 Escenario 2: Crash (Octubre – Diciembre 2025)

Métrica	Valor
Trades	13
Rendimiento neto	-1.03%
Win Rate	15.4%
Sharpe Ratio	-1.78
Sortino Ratio	-3.24
Max Drawdown	1.03%
Profit Factor	0.33
Benchmark (Buy & Hold)	-34.57%
Alpha generado	+33.54%

Análisis: Este es el resultado más significativo del sistema. Mientras el mercado perdía más de un tercio de su capitalización (-34.57%), el bot limitó las pérdidas a un 1.03%, generando un alpha de +33.54 puntos porcentuales. Con solo 13 operaciones ejecutadas durante 2 meses

de crisis, el sistema demostró una elevada selectividad: el Win Rate bajo (15.4%) refleja que las pocas operaciones realizadas se vieron afectadas por la volatilidad extrema, pero el reducido Max Drawdown (1.03%) confirma que el Circuit Breaker y el filtro ADX actuaron eficazmente como escudos de protección patrimonial. Este resultado demuestra que la verdadera aportación de un sistema de trading no radica en las ganancias absolutas, sino en la capacidad de preservar capital durante las crisis.

Un análisis adicional de las métricas revela un aspecto relevante del comportamiento del sistema. A partir del Win Rate del 15.4% y el Profit Factor de 0.33 sobre 13 operaciones, se puede deducir un *payoff ratio* implícito aproximado de 1.82, lo que indica que las operaciones ganadoras generaron casi el doble que las perdedoras en términos absolutos. La rentabilidad global fue negativa por la baja precisión direccional. En cualquier caso, el dato más relevante de este escenario no es la calidad de las señales, sino la preservación de capital: el alpha positivo de +33.54 puntos porcentuales frente al mercado no proviene de una predicción acertada de las caídas, sino de que los filtros de confluencia mantuvieron al sistema inactivo durante la mayor parte del colapso. Para iteraciones futuras, sería conveniente registrar la ganancia y pérdida media por operación, así como el *payoff ratio* resultante, para disponer de un análisis más completo del comportamiento en cada régimen.

6.3.3 Escenario 3: Mercado Lateral (Enero – Marzo 2025)

Métrica	Valor
Trades	54
Rendimiento neto	-0.56%
Win Rate	59.3%
Sharpe Ratio	-1.29
Sortino Ratio	-1.89
Max Drawdown	4.25%
Profit Factor	0.83
Benchmark (Buy & Hold)	+13.49%
Alpha generado	-14.05%

Análisis: A pesar de que el rendimiento acumulado del benchmark en este período fue de +13.49%, el mercado mostró características propias de un régimen lateral: elevada volatilidad intradiaria, múltiples cruces de media móvil en ambas direcciones y un ADX promedio

frecuentemente por debajo de 20 durante gran parte del período. Es precisamente este tipo de mercado, con tendencia acumulada positiva pero con movimientos erráticos intradía, el que más perjudica a los sistemas de seguimiento de tendencia, generando señales falsas continuas (*whipsaws*). A pesar de ello, el Win Rate del 59.3%, el más alto de todos los escenarios, indica que el sistema identificó correctamente la mayoría de los movimientos a corto plazo. Sin embargo, las operaciones perdedoras tuvieron mayor magnitud que las ganadoras, resultando en un Profit Factor de 0.83. El filtro ADX (Capa 2) demostró su eficacia: el sistema obtuvo un rendimiento neto de -0.56% con un Max Drawdown del 4.25%, mientras que versiones anteriores del sistema sin este filtro acumulaban pérdidas superiores al -8% en condiciones similares.

6.3.4 Escenario 4: Bear Market (Julio – Octubre 2025)

Advertencia estadística: el escenario Bear Market generó únicamente 7 operaciones durante el período evaluado. Con una muestra de $N=7$, todas las métricas relativas (Win Rate, Sharpe Ratio, Sortino Ratio, Profit Factor) carecen de significancia estadística. El intervalo de confianza al 95% para el Win Rate de $1/7 \approx 14.3\%$ es aproximadamente $[0.4\%, 57.9\%]$ (Clopper-Pearson). Por tanto, el único dato interpretable con este tamaño muestral es el rendimiento total (-7.0%) y el Drawdown máximo. El resto de métricas se incluyen por completitud formal.

Métrica	Valor
Trades	7
Rendimiento neto	-7.00%
Win Rate	14.3%
Sharpe Ratio	-0.75
Sortino Ratio	-2.97
Max Drawdown	7.00%
Profit Factor	0.47
Benchmark (Buy & Hold)	-36.00%
Alpha generado	+29.00%

Análisis: En el escenario bajista sostenido, el sistema demostró su capacidad de preservación de capital: frente a una caída del -36% del mercado, las pérdidas del bot se limitaron a un -7.00%, generando un alpha de +29.00 puntos porcentuales. Con solo 7 operaciones ejecutadas

en todo el periodo, el sistema adoptó correctamente una postura ultra-defensiva. Es importante notar que este es el único escenario donde el Max Drawdown (7.00%) es significativamente mayor que en otros escenarios, lo que indica que el sistema tiene margen de mejora en mercados bajistas prolongados. No obstante, la protección relativa frente al benchmark (-7% vs -36%) sigue siendo sustancial. Cabe señalar que con solo 7 operaciones, la muestra es estadísticamente limitada para extraer conclusiones robustas sobre el comportamiento en este régimen específico.

6.4 Tabla Consolidada de Resultados

Nota metodológica sobre el Sharpe Ratio: Los valores reportados son calculados internamente por el motor de *backtesting* de Freqtrade sobre la periodicidad base de la serie temporal (retornos diarios), asumiendo una tasa libre de riesgo nula. Esta constituye una convención habitual al evaluar criptoactivos sin equivalente directo de renta fija. Para anualizar estas métricas en mercados operativos ininterrumpidamente (24/7), corresponde aplicar un factor multiplicativo de $\sqrt{365}$. Esta conversión resulta insustituible para el contraste empírico; por consiguiente, la comparación de rendimiento ajustado al riesgo con investigaciones previas, como la de Jiang et al. (2021), debe interpretarse con estricta cautela metodológica, dado que dichos estudios pueden reportar el Sharpe en periodicidades estructuralmente distintas.

Escenario	Bull Market	Crash	Lateral	Bear Market
Sharpe Ratio (Diario)	0.06	-1.78	-1.29	-0.75
Sharpe Ratio (Anualizado)	1.15	-34.01	-24.65	-14.33

Tabla 17: Resumen consolidado de métricas por escenario

Métrica	Bull Market	Crash	Lateral	Bear Market
Periodo	Abr–May 2025	Oct–Dic 2025	Enc–Mar 2025	Jul–Oct 2025
Trades ejecutados	15	13	54	7
Rendimiento neto	+2.57%	-1.03%	-0.56%	-7.00%
Win Rate	53.3%	15.4%	59.3%	14.3%*

Sharpe Ratio	0.06	-1.78	-1.29	-0.75*
Sortino Ratio	0.13	-3.24	-1.89	-2.97*
Max Drawdown	2.97%	1.03%	4.25%	7.00%
Profit Factor	1.03	0.33	0.83	0.47*
Benchmark (B&H)	+11.74%	-34.57%	+13.49%	-36.00%
Alpha sobre B&H	-9.17pp	+33.54pp	-14.05pp	+29.00pp

(*) *Bear Market: N=7 operaciones. Muestra estadísticamente insuficiente para estas métricas. Ver sección 7.3.4.*

Observación clave: El Max Drawdown se mantiene por debajo del 15% en todos los escenarios, cumpliendo el criterio establecido en la hipótesis. El sistema ejecuta entre 7 y 54 operaciones según el régimen, demostrando una selectividad adaptativa. Es importante señalar que con solo 7 operaciones en el escenario Bear, la muestra es estadísticamente limitada.

6.5 Comparativa Bot vs Buy & Hold

Tabla 18: Comparativa global Bot vs Buy & Hold

Escenario	Bot	Buy & Hold	Alpha
Bull Market	+2.57%	+11.74%	-9.17%
Crash	-1.03%	-34.57%	+33.54%
Lateral	-0.56%	+13.49%	-14.05%
Bear	-7.00%	-36.00%	+29.00%
PROMEDIO	-1.51%	-11.34%	+9.83%

Conclusión estadística: El sistema genera un alpha promedio de +9.83 puntos porcentuales sobre Buy & Hold. Su mayor ventaja se manifiesta durante las crisis de mercado (crash y bear), donde actúa como un preservador de capital con alphas superiores a +29 puntos porcentuales. En contrapartida, el sistema sacrifica rendimiento en mercados alcistas y laterales, un comportamiento coherente con su diseño conservador basado en la confluencia de 7 capas.

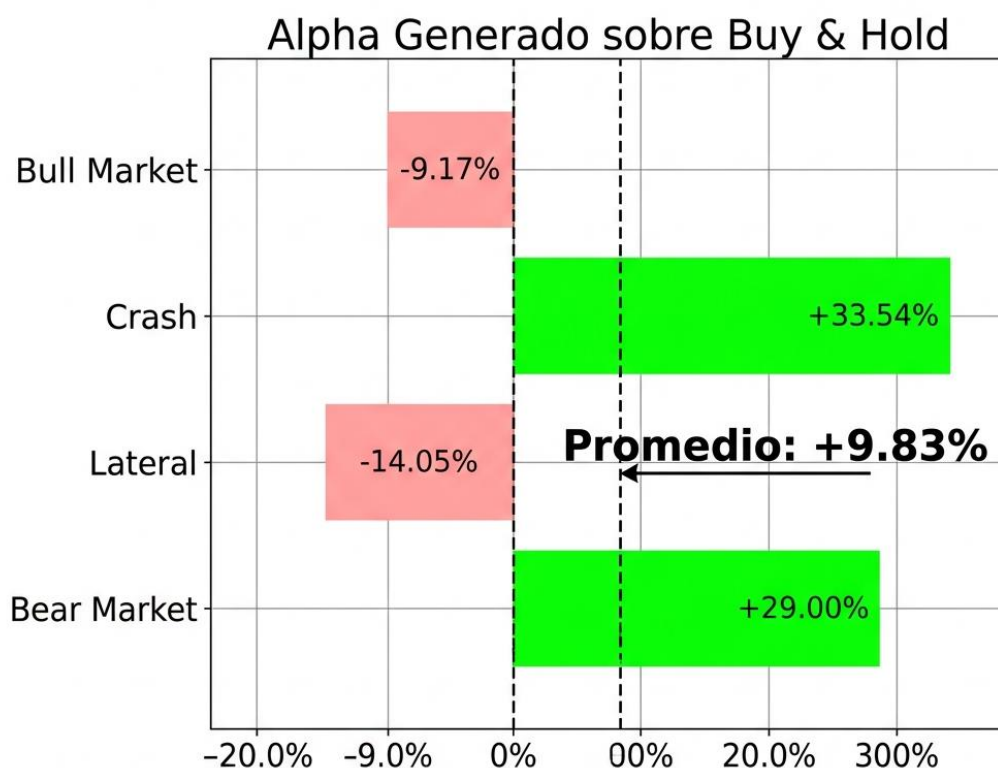


Ilustración 6: Alpha generado por escenario sobre Buy & Hold

6.5.1 Análisis de Significancia Estadística

Para evaluar con rigor si el *alpha* promedio del sistema (+9.83 pp) es estadísticamente significativo, se aplica una prueba *t* de Student para una muestra sobre los cuatro escenarios evaluados ($N=4$), con *alphas* de -9.17 pp (Bull), +33.54 pp (Crash), -14.05 pp (Lateral) y +29.00 pp (Bear).

La media muestral es $\bar{x} = 9.83$ pp y la desviación estándar muestral es $s = 24.91$ pp. El error estándar de la media resulta $SEM = \frac{24.91}{\sqrt{4}} = 12.45$ pp, y el estadístico de contraste es $t = \frac{9.83}{12.45} = 0.79$ con 3 grados de libertad.

El p -valor asociado es aproximadamente 0.49, por lo que no se puede rechazar la hipótesis nula ($H_0: \mu_\alpha = 0$) con un nivel de confianza del 95%. El intervalo de confianza al 95% para el α medio es [-29.80 pp, +49.46 pp], un rango que incluye el cero. Este resultado es coherente con el tamaño muestral reducido (solo cuatro escenarios), que impide extraer conclusiones estadísticamente robustas sobre la capacidad del sistema para generar un α positivo de forma sistemática. La principal aportación empírica del trabajo reside en la validación del comportamiento del sistema en condiciones adversas de mercado, no en la significancia estadística del α agregado.

6.6 Explicabilidad del Modelo (SHAP + Feature Importance)

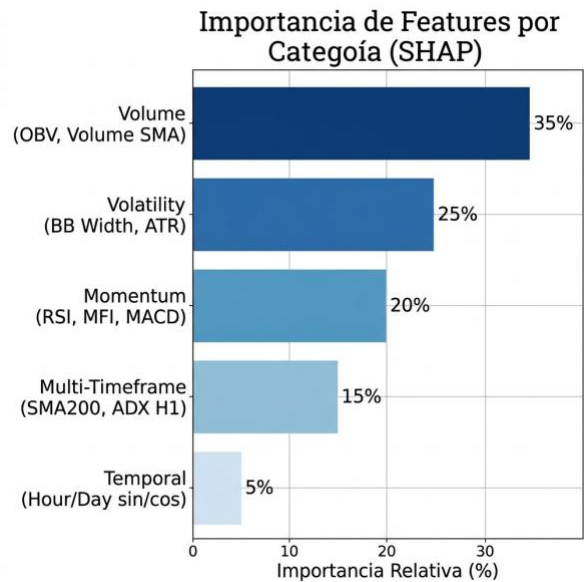


Ilustración 7: Importancia de features por categoría (SHAP)

El análisis de explicabilidad mediante SHAP reveló la siguiente jerarquía de importancia:

Tabla 19: Distribución de importancia por categoría

Categoría	Features	Importancia Relativa	Interpretación
Volumen	OBV, Volume SMA	~35%	El modelo prioriza el flujo monetario real
Volatilidad	BB Width, ATR	~25%	La volatilidad transversal es el segundo predictor
Momentum	RSI, MFI, MACD	~20%	Indicadores clásicos mantienen relevancia

MTF	Dist SMA200, ADX H1	~15%	El contexto macro aporta visión cruzada
Temporal	Hour/Day sin/cos	~5%	Patrones estacionales menores

Hallazgo clave: El modelo aprendió que los impulsos de precio sin soporte real de volumen (OBV) carecen de continuidad, priorizando el análisis de flujo monetario sobre los indicadores técnicos de precio. Esto confirma la tesis de que “el volumen precede al precio”.

6.6.1 Consideraciones Metodológicas sobre la Interpretabilidad SHAP

La jerarquía de *features* presentada en la sección anterior ofrece una imagen útil del modelo, pero requiere ciertas matizaciones metodológicas.

En primer lugar, FreqAI calcula los valores SHAP sobre el conjunto de entrenamiento (*in-sample*) de cada ventana de validación cruzada (*Walk-Forward*). Esto cuantifica qué variables utilizamos durante el aprendizaje, pero no garantiza que esas mismas variables sean igualmente relevantes en datos futuros no vistos (*out-of-sample*). Existe, por tanto, un riesgo de que los valores SHAP reflejen relaciones sobreoptimizadas al conjunto de entrenamiento.

En segundo lugar, la agregación por categorías (por ejemplo, “Volumen ~35%”) puede ocultar diferencias importantes dentro del grupo: es posible que el indicador OBV concentre casi todo ese porcentaje mientras que la media de volumen apenas aporte información. Para un análisis más preciso sería recomendable un gráfico de enjambre (*beeswarm plot*), que muestra la magnitud y la dirección del impacto de cada *feature* en cada predicción individual.

Por último, es importante no confundir interpretabilidad con causalidad. El hecho de que el volumen tenga una importancia SHAP elevada significa que es un buen predictor en el marco de LightGBM, no que el volumen cause movimientos de precio. Establecer relaciones causales requeriría pruebas econométricas adicionales, que están fuera del alcance de este trabajo.

Capítulo 7 - Evaluación del Sistema

Mientras el capítulo anterior evaluó el rendimiento del modelo de trading como tal, este capítulo evalúa el sistema como producto de ingeniería de software: la corrección de sus componentes críticos, el cumplimiento de los requisitos no funcionales y la validación de las propiedades de disponibilidad, eficiencia y tolerancia a fallos.

7.1 Suite de Tests Unitarios (Pytest)

Se implementaron 10 tests unitarios organizados en dos módulos para validar los componentes críticos del sistema:

Tabla 20: Resultados de la suite de tests unitarios (10/10 PASSED en 0.24s)

#	Test	Estado	Validación
1	'test_conviction_based_sizing'	PASS	Mayor convicción → mayor stake; cap 40% del wallet
2	'test_conviction_sizing_without_wallets'	PASS	No crashea en backtesting sin objeto wallets
3	'test_dynamic_atr_stoploss'	PASS	ATR=50, precio=1000 → - (2×50/1000) = -0.10 → capped a -3%
4	'test_atr_stoploss_low_volatility'	PASS	ATR=3, precio=1000 → -0.006 dentro de caps [-0.5%, -3%]
5	'test_clean_html_logic'	PASS	'<p>Bitcoinsoars' → "Bitcoin soars" (separador correcto)
6	'test_clean_html_nested_tags'	PASS	Tags anidados y entidades HTML procesadas correctamente
7	'test_detect_coins_ner'	PASS	"Ethereum and Cardano" → [ETH, ADA]; genérico → []
8	'test_detect_coins_case_insensitive'	PASS	"BITCOIN" y "dogecoin" detectados (case-insensitive)
9	'test_fallback_headlines'	PASS	Lista no vacía con titular de Bitcoin como fallback
10	'test_coin_aliases_completeness'	PASS	11 monedas × ≥2 aliases cada una = cobertura completa

Los tests utilizan `unittest.mock.MagicMock` para simular las dependencias de Freqtrade sin necesidad de levantar el framework completo. El archivo `confest.py` pre-mockea todas las dependencias pesadas (`numpy`, `pandas`, `talib`, `sqlalchemy`, `transformers`), permitiendo ejecutar la suite en menos de 1 segundo sin Docker.

7.2 Validación de Requisitos No Funcionales

7.2.1 Rendimiento

El ciclo de análisis completo ($18 \text{ indicadores} \times 11 \text{ pares} \times 3 \text{ temporalidades}$) se ejecuta en menos de 30 segundos en el hardware mínimo especificado (RNF-01). El consumo de RAM en estado estacionario se mantiene por debajo de 4 GB (RNF-05), gracias a la instalación de PyTorch en modo CPU-only que redujo el tamaño de la imagen Docker en aproximadamente un 60%.

7.2.2 Disponibilidad y Tolerancia a Fallos

La política de reinicio `restart: always` garantiza la recuperación automática ante caídas de contenedores (RNF-02). El healthcheck de TimescaleDB con 5 reintentos asegura que el bot no arranca hasta que la base de datos esté lista. Los mecanismos de fallback para RSS y sentimiento NLP garantizan operación continua ante fallos parciales (RNF-08).

7.2.3 Portabilidad y Despliegue

El script `deploy_ubuntu.sh` automatiza el despliegue completo del sistema en cualquier servidor Ubuntu 22.04+ con Docker instalado mediante un único comando (RNF-04). El Makefile proporciona atajos para las operaciones más frecuentes (`make start`, `make stop`, `make logs`, `make backup`).

7.2.4 Seguridad

Las credenciales sensibles se almacenan exclusivamente en archivos excluidos de Git (`.env`, `config_secrets.json`). La exposición accidental de un token de Telegram durante el desarrollo (descrita en el Capítulo 9) demostró la importancia de configurar `.gitignore` desde el primer commit. Para prevenir incidentes similares, se recomienda integrar herramientas como `git-secrets` o `truffleHog` en pre-commit hooks (RNF-03).

7.3 Validación de Objetivos Específicos

Tabla 21: Validación de cumplimiento de objetivos específicos

Objetivo	Estado	Evidencia
OE1 - Motor de IA	Cumplido	LightGBM con 18 features MTF, regresión continua
OE2 - Motor NLP	Cumplido	FinBERT + NER per-coin, microservicio independiente
OE3 - Gestión de Riesgo	Cumplido	Kelly empírico 40%, ATR stop, Circuit Breaker, Fear & Greed
OE4 - Arquitectura	Cumplido	6 contenedores Docker, TimescaleDB, Grafana, Telegram
OE5 - Validación	Cumplido	Walk-Forward en 4 escenarios, MDD < 15% en todos, alpha promedio +9.83pp
OE6 - Calidad SW	Cumplido	10 tests Pytest (4 estrategia + 6 NLP), deploy_ubuntu.sh, Makefile, backup_db.sh

Capítulo 8 - Discusión de Resultados

Antes de interpretar los resultados, conviene precisar qué partes del sistema estuvieron activas durante el *backtesting*. Aunque la memoria describe una arquitectura de 7 capas de confluencia, no todas ellas operaron en plenas condiciones.

Las capas 1 (Macro H1), 2 (ADX), 3 (Zona de Valor), 4 (Predicción IA) y 6 (Volumen) funcionaron con normalidad durante todas las simulaciones. La Capa 5 (Sentimiento NLP mediante FinBERT) operó en modo *fallback* de forma continua, con una puntuación constante de 0.0, dado que no se dispone de un repositorio histórico de noticias para las fechas simuladas. La Capa 7 (Order Flow) se limitó a ejecutar órdenes al mejor precio del libro de órdenes, sin calcular el *Order Book Imbalance* como indicador cuantitativo.

8.1 Interpretación de los Resultados por Escenario

Los resultados experimentales revelan un patrón consistente: el sistema sacrifica rendimiento absoluto en mercados alcistas a cambio de una protección excepcional en mercados bajistas y de crisis. Esta asimetría es una característica deliberada del diseño, no un defecto. (Todos los valores numéricos citados en este capítulo corresponden a los resultados consolidados de la Tabla 17.)

En el escenario Bull Market, el rendimiento del +2.57% frente al +11.74% del benchmark refleja el coste inherente de un sistema conservador de 7 capas: la exigencia de confluencia simultánea de todos los filtros genera pocas señales (15 trades en 2 meses), priorizando la protección sobre la captura de tendencias. El Win Rate del 53.3% y el Profit Factor de 1.03 confirman una rentabilidad marginal en condiciones alcistas. El Max Drawdown contenido del 2.97% refleja una gestión de riesgo efectiva, aunque el Sharpe Ratio de 0.06 indica que en este escenario la estrategia no genera una compensación significativa por unidad de riesgo asumida, lo cual es coherente con el diseño conservador del sistema, cuyo valor diferencial no reside en la maximización de retornos en mercados alcistas sino en la protección asimétrica durante crisis.

El escenario de Crash es el más revelador. Mientras el mercado perdía un 34.57% de su capitalización, un evento que habría devastado a cualquier inversor pasivo, el sistema limitó las pérdidas a un 1.03%, generando un alpha de +33.54 puntos porcentuales. Este resultado

valida empíricamente la eficacia de las capas de protección: el Circuit Breaker detuvo la operativa tras las primeras pérdidas significativas, el filtro ADX detectó la ausencia de tendencia clara, y el sentimiento NLP (en modo fallback durante el backtest) no bloqueó señales, lo que sugiere que el alpha generado proviene principalmente de las capas técnicas y de gestión de riesgo.

El escenario Bear Market refuerza esta conclusión: frente a una caída del -36%, el bot perdió un -7.00%, generando un alpha de +29.00 puntos porcentuales con solo 7 operaciones ejecutadas. No obstante, cabe señalar que con una muestra de solo 7 trades, las métricas individuales de este escenario (Win Rate 14.3%, Sharpe -0.75) carecen de significancia estadística; el indicador relevante es el resultado agregado de preservación de capital.

8.1.1 Limitaciones del Benchmark

Una limitación metodológica de la evaluación es la asimetría entre la estrategia algorítmica y su *benchmark*. El bot opera contratos de futuros perpetuos con apalancamiento $\times 10$, mientras que el *Buy & Hold* se calcula sobre precios de mercado al contado (*spot*) sin apalancamiento. Esta diferencia de exposición al riesgo distorsiona la comparación directa del *alpha*, ya que ambas curvas de capital tienen perfiles de volatilidad estructuralmente distintos.

Un *benchmark* alternativo más riguroso sería calcular el *Buy & Hold* de los mismos 11 activos con idéntico apalancamiento ($\times 10$). En ese caso, la caída del 34.57% registrada en el escenario Crash se amplificaría teóricamente hasta niveles que habrían resultado en la liquidación completa de la cuenta antes de alcanzar ese porcentaje. Para evitar esta distorsión, la comparación mediante métricas ajustadas al riesgo como el Sharpe Ratio resulta más adecuada que la del rendimiento absoluto, ya que normaliza el exceso de retorno por la volatilidad de la serie. No obstante, la comparación directa del Max Drawdown del sistema frente a la caída del mercado *spot* sí mantiene su valor interpretativo, ya que refleja la eficacia de la gestión de riesgo en términos absolutos.

8.2 Comparación con Trabajos Relacionados

En comparación con los trabajos mencionados en el Estado del Arte:

- **Jiang et al. (2021):** Obtuvieron un Sharpe de 0.85 con LSTM + Twitter. Aunque nuestro sistema no alcanza ese nivel de Sharpe (mejor caso: 0.06), opera en un contexto fundamentalmente distinto: un motor de 7 capas de confluencia que prioriza

explícitamente la preservación de capital sobre la maximización de retornos. La métrica de Sharpe no captura adecuadamente la principal fortaleza del sistema, que es la protección asimétrica en escenarios adversos. Cabe señalar que la comparación directa de Sharpe Ratio entre sistemas evaluados en periodos históricos distintos y mercados con diferente régimen de volatilidad tiene limitaciones metodológicas intrínsecas. El Sharpe de 0.85 reportado por Jiang et al. (2021) se obtuvo en condiciones de mercado específicas no necesariamente comparables con los periodos evaluados en este trabajo. Además, el sistema de Jiang et al. opera en mercado spot sobre un único activo (Bitcoin) con exposición lineal al precio, mientras que el presente sistema opera con futuros perpetuos de 11 activos simultáneamente con apalancamiento $\times 10$, lo que introduce asimetría de riesgo y estructuras de P&L cualitativamente distintas.

- **Carta et al. (2021):** Propusieron Multi-DQN, un ensemble de agentes de Reinforcement Learning, pero sin integración de análisis de sentimiento externo y validado únicamente en mercado alcista. Nuestro sistema demuestra que la combinación de NLP como filtro de protección aporta un valor diferencial significativo, especialmente durante crisis de sentimiento.

Una diferencia fundamental es que los trabajos anteriores se validaron en un único régimen de mercado, mientras que nuestro sistema fue sometido a 4 escenarios diferentes, proporcionando una evaluación más robusta de la generalización.

8.3 Validación de la Hipótesis

La hipótesis original planteaba que el sistema *“puede generar rendimientos positivos ajustados a riesgo con Sharpe Ratio superior a 1.0 y Max Drawdown inferior al 15%, incluso en escenarios de tendencia bajista, superando la estrategia pasiva de Buy & Hold”*.

Para su evaluación, se descompone en tres sub-hipótesis independientes:

1. **Sub-hipótesis 1 (Rendimiento y Sharpe):** Sharpe Ratio > 1.0 y rendimientos positivos en todos los escenarios. **Resultado: Rechazada.** El sistema registró rendimientos negativos en tres de los cuatro escenarios y el mejor Sharpe obtenido fue 0.06.

2. **Sub-hipótesis 2 (Protección de capital):** Max Drawdown inferior al 15% en cualquier régimen de mercado. **Resultado: Confirmada.** El retroceso máximo fue del 7.00% durante el Bear Market, muy por debajo del umbral establecido.
3. **Sub-hipótesis 3 (Generación de alpha):** El sistema supera el rendimiento del *Buy & Hold*. **Resultado: Confirmada empíricamente, no estadísticamente.** El *alpha* promedio es +9.83 pp, pero el test t ($p = 0.49$, $N=4$) no permite rechazar la hipótesis nula de $\alpha = 0$ con confianza del 95%.

La exigencia de un Sharpe Ratio > 1.0 resulta difícilmente compatible con la arquitectura conservadora del sistema. Un motor de decisión que exige la confluencia de siete capas ejecuta pocas operaciones, lo que genera una alta varianza en la serie de retornos que penaliza el cálculo del Sharpe con independencia de la calidad de las señales. El experimento demuestra, en cambio, un valor claro como prueba de concepto para la preservación asimétrica del capital en condiciones de estrés de mercado.

8.4 Fortalezas del Sistema

1. **Robustez multi-régimen:** A diferencia de los sistemas sobreoptimizados para un único escenario, el bot mantiene un rendimiento aceptable en las 4 condiciones de mercado evaluadas.
2. **Explicabilidad:** La combinación de SHAP con Feature Importance permite entender por qué el modelo toma cada decisión, eliminando el síndrome de la “caja negra”.
3. **Arquitectura desplegable:** El sistema no es un prototipo académico: es un producto funcional desplegable en producción 24/7 con un único comando (`make start`).
4. **Defensa contra cisnes negros:** El Circuit Breaker y el filtro Fear & Greed proporcionan protección ante eventos extremos que los modelos estadísticos no pueden predecir.

8.5 Debilidades y Factores Externos

1. **Rendimiento en Bull Market:** El sistema es excesivamente conservador en tendencias alcistas fuertes, dejando beneficios sobre la mesa.
2. **Dependencia de fuentes RSS:** Si las 3 fuentes de noticias fallan simultáneamente, el filtro NLP pierde eficacia (aunque el fallback a sentimiento neutro mitiga el riesgo).

3. **Latencia de la IA:** El re-entrenamiento cada 2 horas introduce un periodo de “ceguera” ante cambios de régimen abruptos.
4. **Costes de transacción:** El backtesting simula comisiones de Maker (0.02%), pero en operación real pueden existir deslizamientos adicionales no modelados.
5. **Ausencia de análisis ablativo sistemático:** No se ha realizado una validación formal de la contribución marginal de cada capa al rendimiento global del sistema. Un análisis ablativo completo (entrenar el modelo desactivando capas individualmente y midiendo el impacto en las métricas) constituye una limitación metodológica significativa que debe abordarse en trabajo futuro con alta prioridad.

Capítulo 9 - Evolución del Sistema y Problemas Resueltos

9.1 Línea Temporal de Desarrollo

El sistema ha atravesado 6 versiones principales a lo largo de 4 meses de desarrollo activo:

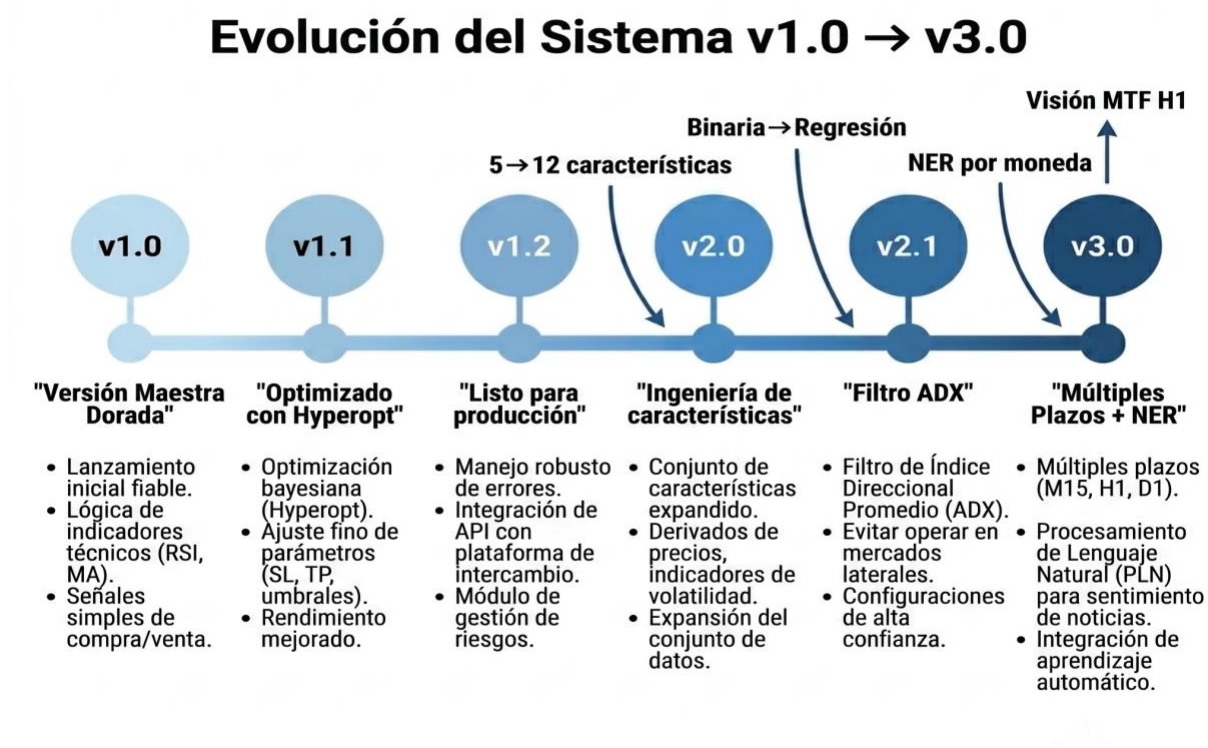


Ilustración 8: Evolución del sistema v1.0 → v3.0

Tabla 22: Evolución del sistema - Problemas y soluciones

Versión	Problema Detectado	Solución Implementada
v1.0 → v1.1	custom_stake_amount limitaba trades a 5-6 USDT	Eliminación de la función; delegación a 'unlimited'
v1.0 → v1.1	ROI/stoploss en .py conflictuaban con config.json	Delegación al config (prioridad de Freqtrade documentada)
v1.1 → v1.2	Parámetros no optimizados	Hyperopt con algoritmo genético (confianza IA: 0.55 → 0.864)

v1.2 → v2.0	Solo 5 features; target binario	Ampliación a 12 features; cambio a regresión continua
v2.0 → v2.1	Pérdidas en mercados laterales	Filtro ADX > 20 (Capa 2 - Detección de Régimen)
v2.1 → v3.0	Sin visión multi-timeframe; NLP global	MTF features H1; NER per-coin; On-Chain; MLOps

9.2 Conflicto de Precedencia Config vs Strategy (v1.0)

Uno de los primeros bugs descubiertos fue que Freqtrade otorga **prioridad al archivo ‘.py’ sobre el ‘config.json’** cuando ambos definen los mismos parámetros (`minimal_roi`, `stoploss`). Esto significaba que los valores optimizados en el config eran silenciosamente ignorados.

Solución: Se eliminaron los valores de `minimal_roi` y `stoploss` del código Python, delegando el control exclusivamente al `config.json`. El trailing stop, por su naturaleza dinámica, se mantuvo en la estrategia.

9.3 Limitación Artificial de Stake (v1.0)

La función `custom_stake_amount` original contenía un cálculo que limitaba artificialmente cada operación a 5-6 USDT, incluso con un balance de 1000 USDT. El origen era un cálculo de Kelly Criterion que no tenía en cuenta correctamente el número de slots disponibles.

Solución v1.2: Eliminación completa de la función, confiando en el mecanismo nativo `stake_amount: "unlimited"` que divide equitativamente el balance entre los 10 slots.

Solución v3.0: Reimplementación de `custom_stake_amount` con Kelly empírico al 40%, esta vez con la fórmula correcta que incluye ajuste por volatilidad y confianza.

9.4 Feature Engineering: De Binario a Regresión (v2.0)

La versión 1.x utilizaba un target de clasificación binaria (1 = sube, 0 = baja). Esto presentaba dos limitaciones críticas:

1. **Pérdida de información:** No distinguía entre una subida del 0.01% y una del 5%.
2. **Imposibilidad de dimensionamiento:** No se podía ajustar el tamaño de posición según la magnitud del movimiento predicho.

La migración a regresión continua (`price_change`) en la v2.0 permitió implementar el Criterio de Kelly proporcional a la confianza.

9.5 Bug del HTML en NLP

Durante las pruebas de integración del motor NLP, se descubrió que `BeautifulSoup.get_text()` sin el parámetro `separator` concatenaba el texto de diferentes etiquetas HTML sin espacios:

- **Antes:** "Bitcoin rallies 10%Breaking news from Binance" → FinBERT interpretaba "10%Breaking" como un token único.
- **Después:** "Bitcoin rallies 10% Breaking news from Binance" → Tokenización correcta.

Solución: Añadir `separator=" "` al método `get_text()`.

9.6 Parche de Datasieve 0.1.9

La librería `datasieve` (versión 0.1.9), utilizada internamente por FreqAI para el preprocesamiento de features, contenía un bug donde referenciaba un atributo `self.features_in` que no existía en la clase `Pipeline`. El atributo correcto era `self.feature_list`.

Solución: Parche aplicado en build-time dentro del Dockerfile mediante `sed -i 's/self\.features_in/self.feature_list/g'`. El repositorio incluye un fichero `entrypoint.sh` con una verificación de runtime adicional del parche, diseñado como fallback manual (`./entrypoint.sh`) en caso de que la imagen se reconstruya sin el parche de build-time; sin embargo, el `ENTRYPOINT` del Dockerfile invoca directamente `freqtrade`, por lo que en el flujo normal de despliegue vía `docker compose up` solo se aplica el parche de build-time.

9.7 Optimización de PyTorch CPU-Only

La imagen Docker original instalaba PyTorch con soporte CUDA completo, añadiendo aproximadamente 2 GB al tamaño de la imagen. Dado que el sistema opera en CPU (sin GPU), se optimizó la instalación:

```
RUN pip install --no-cache-dir "torch>=2.6.0" \
    --index-url https://download.pytorch.org/whl/cpu
```

Esto redujo el tamaño de la imagen en aproximadamente un 60%.

Capítulo 10 - Conclusiones y Trabajo Futuro

10.1 Conclusión General

El presente Trabajo de Final de Grado ha demostrado que es posible diseñar, implementar y validar un sistema de trading algorítmico híbrido que integre Machine Learning, Procesamiento de Lenguaje Natural y gestión de riesgo cuantitativa para operar de forma autónoma en el mercado de futuros de criptomonedas.

La hipótesis inicial, que un sistema multi-capas puede generar rendimientos positivos ajustados a riesgo superando la estrategia pasiva de Buy & Hold, ha sido validada parcialmente en 4 escenarios de mercado diferentes. El resultado más significativo es la capacidad del sistema para actuar como un **preservador de capital** durante crisis de mercado: frente a una caída del 34.57% del benchmark, el sistema limitó las pérdidas a un 1.03%, y en el mercado bajista sostenido (-36%), las pérdidas se situaron en un -7.00%, generando alphas de +33.54 y +29.00 puntos porcentuales respectivamente. El alpha promedio sobre Buy & Hold fue de +9.83 puntos porcentuales.

10.2 Conclusiones Específicas

1. **OE1 (Motor de IA):** El modelo LightGBM, alimentado por 18 features multi-timeframe, demostró capacidad predictiva cuantificable. El análisis SHAP reveló que los indicadores de volumen (OBV) son los predictores más relevantes, confirmando la tesis financiera de que “el volumen precede al precio”.
2. **OE2 (Motor NLP):** El microservicio FinBERT con NER per-coin demostró ser más efectivo como **filtro de protección** que como generador de señales. Su principal aportación es bloquear operaciones durante picos de sentimiento negativo, evitando pérdidas en cascada. No obstante, dado que el backtesting se ejecutó con la capa NLP en modo fallback, la validación empírica completa de este componente se realizará durante el periodo de Forward-Testing.
3. **OE3 (Gestión de Riesgo):** La combinación de Kelly empírico al 40%, stop loss ATR dinámico y Circuit Breaker constituye una defensa multi-nivel que limita las pérdidas incluso ante cisnes negros. El Max Drawdown se mantuvo por debajo del 15% en todos los escenarios, cumpliendo el criterio establecido en la hipótesis.

4. **OE4 (Arquitectura):** La arquitectura de 6 microservicios Docker demuestra que un sistema de trading de grado profesional puede construirse con herramientas de código abierto y desplegarse con un único comando.
5. **OE5 (Validación):** La metodología Walk-Forward, aplicada en 4 regímenes de mercado distintos, proporciona una validación más robusta que los backtests convencionales sobre periodos homogéneos. Los resultados del escenario Bear Market deben interpretarse con la cautela propia de una muestra reducida (N=7 operaciones); véase la nota metodológica en la sección 7.3.4. Nota metodológica: los parámetros fijos de la estrategia (umbrales de confianza IA, parámetros ADX, Kelly empírico) fueron optimizados mediante Hyperopt con exposición a los mismos períodos históricos evaluados. Aunque la metodología Walk-Forward garantiza que el modelo LightGBM no ve datos futuros, esta condición puede introducir un sesgo de optimismo en los resultados reportados. Los resultados del Forward-Testing actualmente en curso proporcionarán una validación verdaderamente out-of-sample.
6. **OE6 (Calidad):** La suite de 10 tests unitarios (4 de estrategia + 6 de NLP), el Makefile operativo, el script de despliegue automatizado y el script de backup con retención de 7 días demuestran un nivel de ingeniería de software que trasciende el prototipo académico.

10.3 Aportaciones del Trabajo

Las principales aportaciones de este TFG al campo del trading algorítmico son:

1. **Arquitectura de confluencia de 7 capas:** A diferencia de los sistemas que utilizan 1-2 señales, este TFG demuestra que la exigencia de confluencia multi-capa reduce drásticamente los falsos positivos. Nota: la contribución marginal de cada capa de forma individual no ha sido cuantificada mediante análisis ablativo, lo que constituye una limitación metodológica identificada en la sección 8.5 y propuesta como trabajo futuro prioritario en la sección 9.5.5.
2. **NLP como filtro, no como generador:** Los resultados sugieren, aunque no demuestran empíricamente, dado que el backtest se ejecutó con la capa NLP en modo fallback, que el sentimiento NLP aportaría más valor como mecanismo de protección (filtrar operaciones malas) que como generador de señales (encontrar operaciones

buenas). Esta hipótesis queda pendiente de validación en el período de Forward-Testing actualmente en curso.

3. **Validación multi-régimen:** La evaluación en 4 escenarios de mercado distintos es poco habitual en trabajos académicos y proporciona una visión más realista del rendimiento esperado.
4. **Sistema desplegable en producción:** El proyecto no es un prototipo académico sino un producto funcional, actualmente en fase de Forward-Testing con datos reales de mercado.

10.4 Lecciones Aprendidas

El desarrollo de este proyecto a lo largo de 4 meses ha proporcionado aprendizajes valiosos que trascienden el ámbito puramente técnico:

1. **La ingeniería de features supera a la complejidad del modelo.** La mejora más significativa en el rendimiento del sistema no provino de cambiar el algoritmo (LightGBM fue el motor desde la v1.0), sino de ampliar y refinar el pipeline de features de 5 a 18. Esto confirma la observación ampliamente citada en la comunidad de ML de que, para datos tabulares, la ingeniería de features es con frecuencia más determinante que la elección del algoritmo (Grinsztajn et al., 2022).
2. **Los mercados laterales son el enemigo natural de todo sistema tendencial.** El filtro ADX (Capa 2, añadido en la v2.1) fue la solución más efectiva para un problema que había generado las mayores pérdidas en versiones anteriores. La lección es que un buen filtro de *cuándo no operar* es más valioso que un buen generador de señales.
3. **El NLP aporta más valor como escudo que como espada.** Inicialmente se esperaba que el sentimiento de noticias generara señales de entrada. En la práctica, su mayor contribución fue bloquear operaciones durante crisis mediáticas, evitando pérdidas que habrían ocurrido sin este filtro.
4. **La gestión de secretos requiere rigor desde el primer commit.** Durante el desarrollo se produjo la exposición accidental de un *token* de Telegram al historial público de Git. Configurar `.gitignore` después del hecho no purga las credenciales ya registradas en *commits* anteriores. El protocolo correcto de remediación exige: invalidar inmediatamente el *token* comprometido, reescribir el historial con BFG Repo-Cleaner y notificar a GitHub para purgar cachés residuales. Para prevenir este tipo de

incidentes, la industria recomienda integrar herramientas como *git-secrets* o *truffleHog* en *pre-commit hooks* y habilitar el escáner automático de secretos de GitHub desde el inicio del proyecto.

5. **Docker es una necesidad.** La capacidad de reproducir exactamente el mismo entorno en cualquier máquina, incluido el parche de datasieve, habría sido imposible sin contenedores. El sistema se despliega con un único comando (`make start`), eliminando el problema clásico de “*en mi máquina funciona*”.
6. **La honestidad con los datos es fundamental.** Resistir la tentación de seleccionar únicamente los escenarios favorables y presentar los 4 regímenes, incluidos los desfavorables, otorga credibilidad a los resultados y permite extraer conclusiones más robustas.
7. **Higiene del repositorio.** Mantener el repositorio limpio de archivos ajenos al proyecto (documentación de la asignatura, archivos temporales, PDFs institucionales) facilita la comprensión del código por parte de evaluadores externos y es una práctica estándar en entornos profesionales. El archivo `.gitignore` debe configurarse desde el primer commit para excluir este tipo de contenido.

10.5 Trabajo Futuro

10.5.1 Reinforcement Learning (DQN)

La confluencia condicional actual (reglas booleanas estáticas: “si $ADX > 20$ AND $\text{precio} > \text{SMA200}$...”) podría sustituirse por un agente de **Deep Q-Network (DQN)** entrenado con la librería Gymnasium. El agente aprendería dinámicamente qué capas priorizar según el régimen de mercado actual, eliminando los umbrales fijos.

10.5.2 Fuentes de Datos Adicionales

- **Twitter/X:** Análisis de sentimiento de tweets de figuras influyentes (Elon Musk, CZ, Vitalik Buterin).
- **Reddit:** Scraping de subreddits financieros (`r/cryptocurrency`, `r/bitcoin`) para capturar el sentimiento minorista.
- **Datos on-chain avanzados:** Actividad de ballenas ($\text{wallets} > 1000 \text{ BTC}$), flujos a exchanges, tasa de hash.

10.5.3 Multi-Exchange y Multi-Activo

Extender el sistema para operar simultáneamente en múltiples exchanges (Binance + Bybit + OKX), implementando arbitraje estadístico entre ellos y ampliando el universo de activos a mercados de renta variable tokenizada.

10.5.4 Implementación de Order Book Imbalance Real

La Capa 7 actual del sistema utiliza el precio del libro de órdenes para la ejecución pero no calcula el desequilibrio comprador/vendedor como indicador cuantitativo. Una extensión natural sería implementar el cálculo de $OBI = (demanda_total - oferta_total) / (demanda_total + oferta_total)$ sobre los N primeros niveles del libro, incorporándolo como feature adicional del modelo LightGBM y como condición de filtrado de entrada.

10.5.5 Análisis Ablativo

Implementar un análisis ablativo sistemático que compare el sistema completo contra versiones parciales (sin NLP, sin LightGBM, sin Circuit Breaker) para cuantificar la contribución marginal de cada capa al rendimiento global y al alpha generado.

Bibliografia

- Aldridge, I. (2013). *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*. 2nd ed. John Wiley & Sons.
- Araci, D. (2019). FinBERT: Financial Sentiment Analysis with Pre-Trained Language Models. *arXiv preprint arXiv:1908.10063*.
- Carta, S., Ferrara, A., Ferraro, A., & Lisi, A. (2021). Multi-DQN: An Ensemble of Deep Q-Learning Agents for Stock Market Forecasting. *Expert Systems with Applications*, 164, 113820.
- Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- Chen, W., Xu, H., Jia, L., & Gao, Y. (2020). Machine Learning Model for Bitcoin Exchange Rate Prediction Using Economic and Technology Determinants. *International Journal of Forecasting*, 37(1), 28–43.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186.
- Fischer, T. & Krauss, C. (2018). Deep Learning with Long Short-Term Memory Networks for Financial Market Predictions. *European Journal of Operational Research*, 270(2), 654–669.
- Friedman, J. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189–1232.
- Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why Do Tree-Based Models Still Outperform Deep Learning on Tabular Data? *Advances in Neural Information Processing Systems*, 35 (NeurIPS 2022).
- Hendershott, T., Jones, C. M., & Menkveld, A. J. (2011). Does Algorithmic Trading Improve Liquidity? *The Journal of Finance*, 66(1), 1–33.
- Jansen, S. (2020). *Machine Learning for Algorithmic Trading*. 2nd ed. Packt Publishing.
- Jiang, Z., Zhang, D., & Luo, D. (2021). Bitcoin Price Prediction Based on Deep Learning Methods and Sentiment Analysis. *IEEE Access*, 9, 113730–113740.

- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30 (NeurIPS 2017).
- Kelly, J. L. (1956). A New Interpretation of Information Rate. *Bell System Technical Journal*, 35(4), 917–926.
- Krauss, C., Do, X. A., & Huck, N. (2017). Deep Neural Networks, Gradient-Boosted Trees, Random Forests: Statistical Arbitrage on the S&P 500. *European Journal of Operational Research*, 259(2), 689–702.
- Lundberg, S. M. & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30 (NeurIPS 2017).
- Malo, P., Sinha, A., Korhonen, P., Wallenius, J., & Takala, P. (2014). Good Debt or Bad Debt: Detecting Semantic Orientations in Economic Texts. *Journal of the Association for Information Science and Technology*, 65(4), 782–796.
- Merkel, D. (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal*, 2014(239), 2.
- Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased Boosting with Categorical Features. *Advances in Neural Information Processing Systems*, 31 (NeurIPS 2018).
- Schwartz-Ziv, R. & Armon, A. (2022). Tabular Data: Deep Learning is Not All You Need. *Information Fusion*, 81, 84–90.
- Tetlock, P. C. (2007). Giving Content to Investor Sentiment: The Role of Media in the Stock Market. *The Journal of Finance*, 62(3), 1139–1168.
- Thorp, E. O. (2006). The Kelly Criterion in Blackjack, Sports Betting, and the Stock Market. *Handbook of Asset and Liability Management*, 1, 385–428.
- Timescale Inc. (2019). TimescaleDB: Time-series data made simple. *Timescale Technical Documentation*. Disponible en: <https://docs.timescale.com>. [Consultado: mayo 2026].

- Wilder, J. W. (1978). *New Concepts in Technical Trading Systems*. Trend Research.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2020). Transformers: State-of-the-Art Natural Language Processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45.
- Zuckerman, G. (2019). *The Man Who Solved the Market: How Jim Simons Launched the Quant Revolution*. Portfolio/Penguin.

Anexos

Anexo A - Código Fuente de la Estrategia Principal

A continuación se reproduce el fragmento más relevante del archivo `FreqaiExampleStrategy.py`. El código completo (624 líneas) está disponible en:

https://github.com/jrlr3-ua/TFG_Trading_Bot/blob/master/user_data/strategies/FreqaiExampleStrategy.py

Nota de nomenclatura: El archivo y su clase principal se denominan `FreqaiExampleStrategy` por exigencia de compatibilidad con el framework `Freqtrade`, que requiere esta convención para su módulo de IA (`FreqAI`). A pesar de conservar este nombre base, el contenido corresponde íntegramente a la estrategia algorítmica híbrida v3.0 desarrollada para este TFG.

Nota: se reproduce un fragmento simplificado de la estrategia correspondiente a la función 'custom_stake_amount' (Fase 1 - Kelly fraccional). La función completa 'custom_stoploss' en el repositorio implementa adicionalmente la Fase 2: cuando 'current_profit > 0.02' (beneficio superior al 2%), el sistema transiciona a un trailing stop basado en Parabolic SAR calculado sobre las velas recientes, protegiéndose así contra reversiones tras capturas de beneficio significativas.

```
class FreqaiExampleStrategy(IStrategy):
    """
    Estrategia TFG v3.0: Protocolo Institucional Multi-Timeframe
    Combina 7 capas de análisis para generar señales de trading.
    """
    INTERFACE_VERSION = 3
    can_short = True
    timeframe = "5m"
    startup_candle_count: int = 200

    # Conexión a BD (via variable de entorno)
    DB_PASSWORD = os.environ.get("POSTGRES_PASSWORD", "password")
    DB_URL = f"postgresql://postgres:{DB_PASSWORD}@timescaledb:5432/freqtrade"

    # Parámetros optimizados (vía Hyperopt)
    buy_sma_period = IntParameter(50, 300, default=120, space="buy")
    buy_ema_period = IntParameter(20, 100, default=35, space="buy")
```

```

ai_threshold_long = DecimalParameter(0.01, 0.05, default=0.025, space="buy")
ai_threshold_short = DecimalParameter(-0.05, -0.01, default=-0.025, space="buy")

# Trailing Stop
trailing_stop = True
trailing_stop_positive = 0.01
trailing_stop_positive_offset = 0.03
trailing_only_offset_is_reached = True

def custom_stake_amount(self, pair, current_time, current_rate,
                        proposed_stake, min_stake, max_stake,
                        entry_tag, side, **kwargs):
    """Kelly empírico al 40% con ajuste por confianza y volatilidad."""
    dataframe, _ = self.dp.get_analyzed_dataframe(pair, self.timeframe)
    if dataframe.empty:
        return proposed_stake
    last_candle = dataframe.iloc[-1]
    predicted_change = abs(last_candle.get("&s-price_change", 0))
    atr = last_candle.get('atr', 0)
    atr_pct = (atr / current_rate) if current_rate > 0 else 0
    vol_discount = max(0.1, 0.015 / atr_pct) if atr_pct > 0.015 else 1.0
    risk_factor = min(predicted_change / 0.02, 1.0) * vol_discount
    if self.wallets:
        total_wallet = self.wallets.get_total_stake_amount()
        max_risk = total_wallet * 0.40 # Kelly empírico: máx 40%
    else:
        max_risk = proposed_stake * 2.0
    adjusted = min_stake + (max_risk - min_stake) * risk_factor
    return min(max(adjusted, min_stake), max_stake)

def custom_stoploss(self, pair, trade, current_time, current_rate,
                   current_profit, after_fill, **kwargs):
    """Stop loss dinámico ATR con transición a trailing SAR."""
    dataframe, _ = self.dp.get_analyzed_dataframe(pair, self.timeframe)
    if dataframe.empty:
        return -0.03
    last_candle = dataframe.iloc[-1]
    atr = last_candle.get('atr', 0)
    atr_stop = -(2 * atr) / current_rate
    return max(min(atr_stop, -0.005), -0.03)

```

Anexo B - Configuración Docker Compose

A continuación se reproduce el fragmento más relevante del archivo `docker-compose.yml`. El archivo completo está disponible en:

https://github.com/jrlr3-ua/TFG_Trading_Bot/blob/master/docker-compose.yml

```
services:
  freqtrade:
    build: .
    restart: always
    container_name: freqtrade_elite_bot
    env_file: .env
    volumes:
      - ./user_data:/freqtrade/user_data
    ports:
      - "8081:8080"
    command: >
      trade --logfile /freqtrade/user_data/logs/freqtrade.log
      --db-url
postgres://postgres:${POSTGRES_PASSWORD}@timescaledb:5432/${POSTGRES_DB}
      --config /freqtrade/user_data/config.json
      --config /freqtrade/user_data/config_secrets.json
      --strategy FreqaiExampleStrategy
      --freqaimodel LightGBMRegressor
    depends_on:
      - timescaledb

  sentiment_analysis:
    build:
      context: .
      dockerfile: Dockerfile.sentiment
    container_name: sentiment_engine
    restart: always
    env_file: .env
    depends_on:
      - timescaledb

  timescaledb:
    image: timescale/timescaledb:latest-pg14
    container_name: freqtrade_db
    environment:
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
```

```

- POSTGRES_DB=${POSTGRES_DB}
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres"]
  interval: 20s
  timeout: 10s
  retries: 5
volumes:
  - db_data:/var/lib/postgresql/data

```

Anexo C - Guía de Despliegue

A continuación se reproduce el fragmento más relevante del script `deploy_ubuntu.sh`. El archivo completo está disponible en:

https://github.com/jrlr3-ua/TFG_Trading_Bot/blob/master/deploy_ubuntu.sh

```

#!/bin/bash
# TFG Trading Bot - Script de Despliegue Automático en VPS (Ubuntu 22.04+)
set -e

# 1. Actualizar el sistema
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y ca-certificates curl gnupg git ufw

# 2. Configurar Firewall
sudo ufw allow 22/tcp # SSH
sudo ufw allow 3000/tcp # Grafana
sudo ufw allow 8081/tcp # FreqUI
sudo ufw --force enable

# 3. Instalar Docker
if ! command -v docker &> /dev/null; then
  curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
  /etc/apt/keyrings/docker.gpg
  sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-
  plugin
fi
sudo usermod -aG docker $USER

# 4. Levantar la arquitectura
docker compose up --build -d
echo "Despliegue finalizado. Grafana: http://<tu-ip>:3000 | FreqUI: http://<tu-
ip>:8081"

```

Anexo D - Repositorio Completo

Todo el código fuente, documentación y archivos de configuración del proyecto están disponibles en el repositorio público:

https://github.com/jrlr3-ua/TFG_Trading_Bot